

SMARTGOV MARKET

An Integrated E-Governance and E-Commerce Platform

Project Documentation / Software Project Report

ACADEMIC PROJECT SUBMISSION RECORD

A Comprehensive Final Year Software Engineering Project Report Submitted in Partial Fulfillment of the Degree of Bachelor of Science in Computer Science & Information Technology (B.Sc. CSIT) / Bachelor of Computer Engineering.

Candidate / Student Name:	[LEAVE PLACEHOLDER - Student Name]
Roll / Registration No.:	[LEAVE PLACEHOLDER - Roll / Registration No.]
Academic Program / Subject:	[LEAVE PLACEHOLDER - B.Sc. CSIT / B.E. Computer / BCA]
Department:	[LEAVE PLACEHOLDER - Department of Computer Science & Information Technology]
College / University:	[LEAVE PLACEHOLDER - University / College Name]
Project Supervisor:	[LEAVE PLACEHOLDER - Project Supervisor / Lecturer]
Academic Session / Year:	[LEAVE PLACEHOLDER - e.g., 2025/2026]

CERTIFICATE OF APPROVAL

This is to certify that the project entitled "SMARTGOV MARKET: An Integrated E-Governance and E-Commerce Platform", submitted by [LEAVE PLACEHOLDER - Student Name] (Roll/Reg No: [LEAVE PLACEHOLDER - Roll / Registration No.]) in partial fulfillment of the requirements for the degree of [LEAVE PLACEHOLDER - Course/Subject] to the [LEAVE PLACEHOLDER - Department], [LEAVE PLACEHOLDER - College/University], is a bonafide record of independent software development work carried out under my supervision and guidance.

The results embodied in this report have been verified against the actual system implementation and have not been submitted to any other university or institute for the award of any degree or diploma.

[LEAVE PLACEHOLDER - Supervisor] [LEAVE PLACEHOLDER - Head of Dept]
Project Supervisor Head of Department
Department of Computer Science Department of Computer Science
Date: _____ Date: _____

CANDIDATE DECLARATION

I hereby declare that this project report entitled "SMARTGOV MARKET: An Integrated E-Governance and E-Commerce Platform" is an authentic record of my own work conducted under the supervision of [LEAVE PLACEHOLDER - Supervisor], [LEAVE PLACEHOLDER - College/University].

I confirm that:

1. The software codebase, architectural designs, database schemas, and documentation presented herein reflect the actual working system implemented for this project.
2. The work has not been previously submitted in part or full for the award of any other academic degree, diploma, or certificate.
3. Proper acknowledgements, citations, and references have been provided for all external libraries, packages, and frameworks utilized during development.

[LEAVE PLACEHOLDER - Student Name]

Roll / Reg. No.: [LEAVE PLACEHOLDER]

Department: [LEAVE PLACEHOLDER]

Date: _____

ACKNOWLEDGEMENT

I would like to express my deepest gratitude and sincere appreciation to my project supervisor, [LEAVE PLACEHOLDER - Supervisor], whose invaluable guidance, constructive criticism, and continuous encouragement made the successful design and implementation of the SmartGov Market platform possible.

I am profoundly indebted to the faculty members and staff of the [LEAVE PLACEHOLDER - Department], [LEAVE PLACEHOLDER - College/University] for providing the academic foundation, computing laboratories, and technical resources necessary for conducting this project.

Special thanks are extended to the open-source software communities responsible for PHP, MariaDB/MySQL, Bootstrap 5, Leaflet.js, OpenStreetMap, and Chart.js, without whose robust software foundations this integrated e-governance and e-commerce platform could not have been engineered.

Finally, I express my heartfelt gratitude to my family and fellow peers for their unwavering moral support, patience, and encouragement throughout the entire duration of this project.

[LEAVE PLACEHOLDER - Student Name]

ABSTRACT

The rapid proliferation of digital technologies has transformed governance and retail individually; however, a deep divide persists between citizen-centric municipal administration and local economic commerce. Traditional local businesses face cumbersome, paper-heavy verification workflows to obtain operating licenses, while citizens must navigate disjointed portals for municipal grievances, licensing status, and verified local shopping.

To bridge this operational gap, this report presents the design, architecture, and implementation of "SmartGov Market" (locally branded as HATIYA - Digital Governance & Marketplace Platform), an integrated web platform that unifies municipal e-governance with a verified local e-commerce marketplace into a cohesive single-tenant digital ecosystem. Built on a resilient PHP 8.2+ backend with a normalized MariaDB relational database (25 tables), the platform provides four distinct Role-Based Access Control (RBAC) tiers: System Administrator, Municipal Government Officer, Verified Local Vendor, and Citizen/Customer.

The E-Governance subsystem automates the entire commercial licensing lifecycle. Prospective vendors submit multi-step digital applications, attach official compliance files (PAN/VAT certificates, citizenship cards, municipal trade permits), and define their store location on an interactive Leaflet/OpenStreetMap GIS map. Government inspection officers audit submitted records via an inspection dashboard, review individual files, issue correction requests, and upon approval, trigger the automated generation of a cryptographically traceable Digital Business License (LIC-YYYY-XXXXXX). Each license features a dynamic QR code linked to a public real-time verification endpoint (`verify.php`). Furthermore, the governance module features a public grievance system (CMP-YYYY-XXXXXX) that supports geographic pinpointing, priority assignment, evidence uploads, and departmental dispatch.

The E-Commerce subsystem restricts marketplace listing privileges strictly to approved, verified vendors possessing an active business license. Citizens browse verified local goods with category filtering, manage persistent carts, and execute checkout transactions with server-side stock and price validation. Payment processing supports native Cash on Delivery (COD) and a simulated gateway interface for national digital wallets (eSewa, Khalti, ConnectIPS) designed specifically for demonstration and academic evaluation without requiring live banking credentials. An atomic database transaction protocol guarantees inventory consistency and triggers immediate real-time in-app notifications and audit log records. A verified-buyer review system ensures feedback authenticity.

Comprehensive testing—including unit, integration, role authorization, and security vulnerability evaluations—confirmed high reliability, sub-second query responsiveness, strict protection against SQL injection and Cross-Site Request Forgery (CSRF), and reliable bilingual support (English and Nepali). The resulting platform demonstrates that integrating municipal governance workflows directly with localized digital commerce enhances administrative transparency, eliminates unlicensed commercial activity, fosters citizen trust, and empowers local economic growth.

LIST OF ABBREVIATIONS

Acronym	Full Technical Term
API	Application Programming Interface
BCRYPT	Blowfish-based Cryptographic Hashing Algorithm
COD	Cash on Delivery
CRUD	Create, Read, Update, Delete
CSRF	Cross-Site Request Forgery
CSS	Cascading Style Sheets
DBMS	Database Management System
DFD	Data Flow Diagram
DOM	Document Object Model
ER / ERD	Entity-Relationship / Entity-Relationship Diagram
GIS	Geographic Information System
HTML	HyperText Markup Language
HTTP / HTTPS	HyperText Transfer Protocol / HyperText Transfer Protocol Secure
i18n	Internationalization / Multilingual Localization
JSON	JavaScript Object Notation
LAN	Local Area Network
MIME	Multipurpose Internet Mail Extensions
MVC	Model-View-Controller
MySQL / MariaDB	My Structured Query Language Relational Database Engine
PAN	Permanent Account Number
PDO	PHP Data Objects
PHP	Hypertext Preprocessor
QR Code	Quick Response Code
RBAC	Role-Based Access Control
SQL	Structured Query Language
UAT	User Acceptance Testing
UI	User Interface
URI / URL	Uniform Resource Identifier / Uniform Resource Locator
VAT	Value Added Tax
XAMPP	Cross-Platform, Apache, MariaDB, PHP, and Perl
XSS	Cross-Site Scripting

CHAPTER 1: INTRODUCTION

1.1 Background

Electronic Governance (E-Governance) and Electronic Commerce (E-Commerce) represent two of the most transformative digital paradigms of the modern era. E-Governance refers to the deployment of Information and Communication Technologies (ICT) by governmental agencies to enhance administrative efficiency, foster transparency, streamline public service delivery, and empower citizens with direct civic participation. Simultaneously, E-Commerce encompasses the digital transaction of goods and services over computer networks, eliminating geographic barriers and democratizing market access for businesses of all scales.

Historically, these two domains have developed within distinct institutional and technological silos. Municipal e-governance initiatives typically focus on regulatory compliance, digital tax collection, building permit approvals, and civic grievance resolution. Conversely, commercial online marketplaces focus strictly on inventory aggregation, consumer engagement, digital payments, and logistics fulfillment. This functional partition forces local merchants to interact with governmental regulatory boards through disjointed bureaucratic procedures while separately navigating commercial platforms that offer zero municipal verification or institutional credibility.

The convergence of E-Governance and E-Commerce introduces a symbiotic digital framework. When local governance authorities digitize merchant licensing and couple it directly with a municipal digital marketplace, the administrative barrier between regulatory oversight and local economic vitality is dissolved. Local merchants receive formal legitimacy through authenticated digital licensing, municipal authorities maintain real-time visibility into commercial activity, and citizens gain access to a trusted marketplace where all participating vendors are verified public entities.

1.2 Introduction to SmartGov Market

SmartGov Market (locally operationalized and branded under the title "HATIYA - Digital Governance & Marketplace Platform", referencing the traditional weekly community market gathering) is a full-stack, integrated web application engineered to bridge municipal governance and digital commerce into a single unified platform.

SmartGov Market is not a mere marketplace clone, nor is it simply a municipal static noticeboard. Rather, it represents an integrated digital ecosystem where municipal regulatory oversight directly underpins commercial transaction legitimacy. Within this platform, local vendors register and submit multi-stage business applications accompanied by digital copies of official government credentials (such as PAN/VAT registrations, citizenship identification cards, and municipal operating permits). Municipal Government Officers review and verify these credentials through an administrative workflow, culminating in the issuance of an official, cryptographically traceable Digital Business License (bearing the standard serial format LIC-YYYY-XXXXXX).

Each digital business license is equipped with an authenticated Quick Response (QR) code linked to a real-time public verification portal (verify.php). Critically, the platform enforces strict architectural boundaries: only vendors possessing an active, verified digital license are granted listing and sales privileges within the marketplace. Citizens and customers are provided with a modern, glassmorphic marketplace interface where they can browse authenticated local products, manage carts, place atomic orders with stock reservation, select delivery addresses, and execute transactions using Cash on Delivery (COD) or simulated national digital wallet gateways (eSewa, Khalti, and ConnectIPS). Furthermore, citizens can lodge location-aware public grievances (CMP-YYYY-XXXXXX) mapped on an interactive Geographic Information System (GIS) Leaflet map, track service applications, and receive real-time dashboard notifications.

1.3 Problem Statement

Contemporary local economic and municipal administrative systems suffer from several critical shortcomings:

1. **Fragmented Business Registration and Licensing:** Traditional municipal business licensing relies on physical paperwork, in-person queueing, and manual file reviews. This leads to substantial processing delays, administrative overhead, and the absence of a unified, publicly verifiable registry of operating licenses.
2. **Inability to Verify Merchant Legitimacy:** On conventional open commercial platforms, consumers frequently encounter fraudulent sellers, counterfeit merchandise, and misleading business representations due to the absence of verifiable governmental accreditation.
3. **Lack of Transparent Grievance Mechanisms:** Citizens facing consumer exploitation, substandard product quality, or service deficiencies encounter cumbersome municipal complaint procedures lacking real-time progress tracking, evidence archiving, or geographic mapping.
4. **Economic Disadvantage for Small Local Merchants:** Micro, small, and medium enterprises (MSMEs) in local municipalities often lack the technical resources to establish standalone digital storefronts, leaving them vulnerable to predatory intermediate commercial aggregators.
5. **Ineffective Administrative Oversight:** Municipal authorities lack real-time data analytics regarding local commercial activity, registered vendors, market revenue flows, and consumer complaint hotspots.

1.4 Motivation

The development of SmartGov Market was driven by the imperative to modernize municipal governance while actively stimulating local commerce. In many developing economies, the local government represents the most immediate tier of civic trust. By leveraging this institutional trust as the foundation for digital commerce, several transformative advantages emerge:

- **Empowering Municipalities:** Equipping local ward and municipal offices with web-based tools to process business applications, inspect digital documents, enforce statutory compliance, and issue instant digital licenses.
- **Cultivating Consumer Confidence:** Assuring citizens that every vendor listed on the platform is an officially vetted, legally compliant merchant with physical operating coordinates.
- **Streamlining Civic Redressal:** Integrating public complaints directly into the municipal officer dashboard with geographic coordinates to expedite dispute resolution.
- **Technical Advancement:** Demonstrating how modular PHP architecture, relational database normalization, dynamic client-side scripting, and geographic information systems can solve real-world governance challenges without prohibitive proprietary software licensing costs.

1.5 Objectives of the Project

The project is guided by overarching general objectives complemented by specific technical and functional targets:

1.5.1 General Objective:

To design, engineer, test, and document a robust, secure, and integrated web-based platform that harmonizes municipal regulatory licensing (E-Governance) with a verified commercial retail marketplace (E-Commerce) for local municipalities.

1.5.2 Specific Objectives:

- To implement a multi-tiered Role-Based Access Control (RBAC) system supporting Administrator, Government Officer, Vendor, and Citizen/Customer roles.
- To develop an automated vendor licensing lifecycle featuring multi-step digital document submission, officer document inspection, correction requests, and instant cryptographic license generation (LIC-YYYY-XXXXXX).
- To construct a public, mobile-accessible QR verification engine (verify.php) enabling instant authentication of merchant licensing status (VALID, EXPIRED, SUSPENDED).
- To engineer a full-featured e-commerce marketplace incorporating product categorization, keyword search, inventory stock management, persistent shopping carts, and atomic order placement transactions.
- To integrate a multi-method payment architecture supporting native Cash on Delivery (COD) alongside an academic simulated gateway interface for eSewa, Khalti, and ConnectIPS.
- To incorporate an interactive Leaflet.js / OpenStreetMap GIS mapping module to visually plot verified vendor

store locations and public grievance coordinates.

- To implement a dual-scope citizen grievance redressal system (CMP-YYYY-XXXXXX) with evidence file attachment and municipal officer dispatch.
- To enforce verified-purchaser product reviews, ensuring that feedback is restricted exclusively to customers who have taken delivery of the item.
- To integrate bilingual localization (i18n) supporting seamless switching between English and Nepali languages.
- To establish a comprehensive security audit logging framework (audit_logs) tracking user activities, IP addresses, and state alterations across the application.

1.6 Scope of the Project

The scope of SmartGov Market encompasses the complete functional pipeline from municipal vendor onboarding to end-user retail fulfillment:

- User Onboarding & Authentication: Secure citizen registration, vendor onboarding, session management, BCRYPT password hashing, and password recovery.
- Administrative Governance: Centralized administrative and municipal officer dashboards for inspecting applications, approving or rejecting documents, managing municipal service directories, and analyzing visual Chart.js metrics.
- Licensing Subsystem: Automated creation and printable presentation of official business certificates complete with machine-readable QR codes and validity periods.
- E-Commerce Engine: Vendor product catalog management, image uploads, stock inventory decrementing, customer cart operations, checkout processing, order status tracking (Pending Payment, Paid, Processing, Shipped, Delivered, Cancelled), and customer transaction logs.
- Public Complaints & Municipal Services: Citizen application for official municipal services (permits, recommendations), grievance filing with photo/PDF attachment, and status tracking.
- Visual & Geographic Tools: Leaflet GIS map visualization for stores and complaints; dynamic Chart.js dashboards for sales revenue, vendor distribution, and complaint categories.

1.7 Limitations of the System

Based on the actual implementation of the project, the following realistic boundaries and limitations are explicitly noted:

1. Simulated Payment Gateway: In accordance with academic demonstration and local testing constraints, integrations with eSewa, Khalti, and ConnectIPS are implemented as simulated demo gateways (demo-payment.php). While state transitions and transaction IDs (DEMO-XXXXXXXX) are stored atomically in the database, no real financial debits occur against live bank accounts.
2. Localized SMS/Email Dispatch: In-app notifications are delivered dynamically to authenticated user dashboards; however, external third-party SMS/SMTP gateway dispatch requires active carrier credentials and API credits, which are not bound in this local deployment.
3. Server Architecture: The current release is architected as a modular monolithic PHP web application running under Apache/PHP with MariaDB in a local environment (such as XAMPP), rather than a distributed cloud microservices infrastructure.
4. Document Verification Automation: Document inspection (PAN/VAT cards, identity certificates) is performed manually by human municipal officers via the administrative document viewer rather than through automated optical character recognition (OCR) or live national ID database API sync.

1.8 Significance of the Project

SmartGov Market provides distinct, tangible benefits across all stakeholder groups:

- For Citizens / Customers: Guarantees peace of mind by ensuring every merchant on the platform is government-verified; provides direct access to municipal services and public grievance channels; delivers a seamless, localized shopping experience with transparent order tracking.

- For Local Vendors: Drastically accelerates the business licensing process; provides an official digital business license with a printable certificate and QR verification code; offers an immediate digital storefront to reach local consumers without intermediate aggregator commission fees.
- For Municipal Officers: Replaces physical paper queues with an organized digital inspection inbox; allows granular inspection and remarking on compliance documents; automates license generation; provides geographic visibility over complaints.
- For System Administrators: Grants comprehensive oversight over system users, vendor statuses, marketplace transactions, security audit logs, and instant CSV analytical report generation.

1.9 Organization of the Report

This report is systematically organized into nine chapters and supplementary appendices:

- Chapter 1: Introduction establishes the background, concept, problem statement, objectives, scope, limitations, and significance of the SmartGov Market platform.
- Chapter 2: Literature Review & Existing System surveys traditional municipal procedures and standalone e-commerce systems, detailing their deficiencies and presenting a rigorous comparative matrix.
- Chapter 3: System Analysis and Requirements details the functional and non-functional requirements, user personas, hardware/software specifications, and multi-dimensional feasibility study.
- Chapter 4: System Design provides comprehensive architectural designs, including system architecture diagrams, RBAC matrices, Use Case diagrams, Data Flow Diagrams (DFDs), Entity-Relationship (ER) models, complete 25-table database schemas, sequence diagrams, and activity diagrams.
- Chapter 5: System Implementation details the technical construction of the platform, including development stack, folder structure, database connection logic, authentication, security countermeasures, and module workflows.
- Chapter 6: User Interface and Screenshots presents visual layouts and screen documentation for over twenty distinct operational interfaces.
- Chapter 7: Testing and Quality Assurance details testing strategies, unit/integration verification, and a comprehensive functional test case matrix documenting actual pass/fail statuses.
- Chapter 8: Results and Discussion analyzes the operational achievements of the platform across e-governance, commerce, and administrative domains, alongside challenges encountered.
- Chapter 9: Conclusion and Future Enhancements synthesizes key conclusions and identifies clear technical trajectories for future enhancements.
- References & Appendices provide academic citations, complete database schemas, source code listings, setup instructions, and user manuals.

CHAPTER 2: LITERATURE REVIEW / EXISTING SYSTEM

2.1 Introduction

The literature surrounding public administration and electronic commerce highlights a significant paradigm shift over the past two decades. As governments globally adopt 'Digital First' strategies, the definition of public service has expanded from passive informational websites to interactive transactional portals. Concurrently, digital commerce has matured from basic mail-order catalogs into hyper-connected retail marketplaces. This chapter examines existing paradigms in both sectors, evaluates traditional operational workflows, and establishes the theoretical and practical justification for the integrated SmartGov Market architecture.

2.2 Existing E-Governance Systems

Modern e-governance implementations are typically categorized into four delivery models: Government-to-Citizen (G2C), Government-to-Business (G2B), Government-to-Government (G2G), and Government-to-Employee (G2E). In developing countries, municipal G2C and G2B portals primarily deliver informational circulars, downloadable PDF application forms, tax payment portals, and rudimentary grievance filing forms.

While platforms such as national single-window registries have streamlined company incorporation at the macro-level, municipal-level trade licensing remains largely disconnected. Local businesses frequently obtain municipal operational permits through manual physical submissions. Furthermore, public verification of these licenses is virtually non-existent: third parties have no immediate, mobile-accessible mechanism to verify whether a local vendor's operating certificate is valid, suspended, or forged.

2.3 Existing E-Commerce Systems

The global e-commerce landscape is dominated by large-scale enterprise marketplaces (such as Amazon, eBay, and regional aggregators like Daraz in South Asia). These platforms excel at catalog aggregation, search optimization, recommendation algorithms, customer loyalty incentives, and logistical fulfillment.

However, enterprise marketplaces operate on commercial profit incentives that prioritize large national distributors and high-volume importers over localized community artisans and municipal producers. More critically, vendor onboarding on these platforms focuses solely on internal marketplace compliance (tax identification and banking credentials) without verifying local municipal operating legality or physical municipal zoning compliance. Consequently, consumer trust suffers when counterfeit goods or unregulated vendors infiltrate the marketplace.

2.4 Problems in Existing / Traditional Systems

A rigorous analysis of traditional municipal administration and conventional e-commerce reveals several structural deficiencies:

1. **Information Asymmetry:** Consumers have no mechanism to ascertain whether an online seller operates with valid municipal trade approval, certified hygiene standards, or verified physical operating premises.
2. **Inefficient Licensing Overhead:** Traditional vendor verification requires municipal officers to process paper archives, leading to misplaced files, delayed business openings, and lost municipal tax revenue.
3. **Lack of Counterfeit Protection for Licenses:** Paper certificates displayed on shop walls are easily falsified, altered, or presented long past their expiration date without public detection.
4. **Disjointed Grievance Redressal:** When consumers encounter fraudulent practices or defective products, traditional municipal consumer protection units require in-person formal petitions, discouraging citizen reporting.
5. **High Barrier to Entry for MSMEs:** Micro and small local enterprises find established commercial marketplaces cost-prohibitive due to commission fees, complex logistics requirements, and algorithmic bias favoring international imports.

2.5 Proposed SmartGov Market System

The SmartGov Market platform resolves these systemic problems by integrating municipal regulatory licensing directly into an accessible local e-commerce marketplace:

- Unified Onboarding Pipeline: Local businesses complete registration, upload compliance documentation, and pin their physical store coordinates on an interactive GIS map in a single digital session.
- Rigorous Officer Verification Workflow: Dedicated municipal officer portals present digital document viewers, remark fields, correction request mechanisms, and single-click approval protocols.
- Cryptographically Traceable Digital Licenses: Automated generation of standardized digital licenses (LIC-YYYY-XXXXXX) embedded with authenticated QR codes that link to a public, mobile-friendly verification endpoint.
- Exclusive Marketplace Listing Privilege: The marketplace catalog strictly excludes unapproved or suspended merchants. Only vendors holding a verified, active license can publish products, receive customer orders, and collect payments.
- Transparent Citizen Grievance Redressal: Integrated public complaint portal featuring Leaflet GIS geocoding, file evidence uploads, automated departmental dispatch, and real-time status updates.

2.6 Comparison Between Existing and Proposed Systems

To clearly illustrate the architectural and operational advantages of SmartGov Market, the table below provides a detailed multi-criteria comparative analysis against traditional municipal processes, generic commercial marketplaces, and standalone e-governance portals.

2.7 Related Technologies and Concepts

The conceptual design of SmartGov Market is informed by established software engineering and public administration frameworks:

- Relational Database Normalization: Structuring database entities into Boyce-Codd / Third Normal Form (3NF) to eliminate data redundancy, maintain referential integrity, and enforce strict foreign key constraints across commercial and civic tables.
- Role-Based Access Control (RBAC): Enforcing granular authorization boundaries where users are assigned roles with strictly defined permission scopes (Admin, Officer, Vendor, Customer).
- Geographic Information Systems (GIS) in Municipal Governance: Utilizing spatial geocoding and map visualizers (Leaflet.js with OpenStreetMap) to contextualize civic grievances and merchant distributions across municipal wards.
- Cryptographic Token Verification: Utilizing randomized unique tokens and formatted alphanumeric identifiers for secure public verification and password recovery without exposing internal database keys.

Table 2.1: Comparison Between Existing and Proposed Systems

Metric	Traditional Office	Generic E-Com	SmartGov Market
Merchant Registration	Manual Paper Forms & Queues	Self-Service (Tax ID Only)	Multi-Step KYC + GIS Coordinates
Document Verification	Manual Physical Auditing	Internal Marketplace Review	Granular Digital Viewer & Checklist
Business Licensing	Paper Certificate (Forgible)	None Issued	Cryptographic Code (LIC-YYYY-XXXXXX)
Public Verification	Impossible in Real-Time	Not Applicable	Instant Mobile QR Portal (verify.php)
Marketplace Listing	None (Physical Stall)	Open to Any Seller	Strictly Restricted to Verified Licensees
Ordering & Inventory	Manual Cash Flow	Atomic Cart & Decrement	Atomic DB Transactions + Stock Protection
Payment Processing	Cash Only	Cards, Wallets, COD	Native COD + Simulated Wallet Gateways
Civic Grievance	Formal Paper Petition	Seller Dispute Ticket	Interactive Leaflet GIS Map + Photo Evidence
Review Authenticity	Word of Mouth	Open (Vulnerable to Spam)	Restricted to Verified Delivered Buyers
Audit Trail	Physical Archive Binders	Private Proprietary Logs	Searchable Immutable Audit Trail

CHAPTER 3: SYSTEM ANALYSIS AND REQUIREMENTS

3.1 System Analysis

System analysis constitutes the systematic investigation of user needs, administrative workflows, data flows, and technological constraints required to design a dependable integrated web platform. The analysis phase of SmartGov Market evaluated the interactions among four primary user groups: Citizens/Customers seeking reliable civic services and authentic goods; Local Merchants seeking digital licensing and direct commercial sales; Municipal Officers charged with document inspection and regulatory enforcement; and System Administrators responsible for platform integrity, security, and reporting.

3.2 Functional Requirements

Functional requirements define the core software capabilities, behavioral responses, and processing logic implemented within the SmartGov Market codebase. Based exclusively on the actual implementation, the functional requirements are categorized by functional module:

3.2.1 User & Authentication Management:

- FR-01: The system shall provide user registration for Citizens and Vendors with input validation (full name, unique email, phone number, physical address, municipality, district).
- FR-02: The system shall enforce BCrypt password hashing on all user passwords prior to database persistence.
- FR-03: The system shall provide secure authentication (login/logout) with session fixation protection (session_regenerate_id) and role-based dashboard routing.
- FR-04: The system shall enforce automatic session timeout after 7,200 seconds (2 hours) of user inactivity.
- FR-05: The system shall provide self-service password recovery via secure, time-expiring cryptographic tokens (password_resets).

3.2.2 Vendor Onboarding & Digital Licensing (E-Governance):

- FR-06: The system shall provide a multi-step vendor registration form capturing business name, business sector, PAN/VAT number, registration number, address, and GIS map coordinates.
- FR-07: The system shall allow vendors to upload official compliance documents (PAN/VAT certificates, citizenship cards, municipal permits) restricted to PDF, JPG, and PNG formats under 5MB.
- FR-08: The system shall maintain vendor status states: 'Pending', 'Under Review', 'Correction Required', 'Approved', 'Rejected', and 'Suspended'.
- FR-09: The system shall restrict unapproved or suspended vendors from adding products or publishing catalog items to the marketplace.
- FR-10: The system shall provide Government Officers with an inspection queue to review applications, view uploaded files in a secure viewer, verify or reject individual files, and append officer remarks.
- FR-11: The system shall automatically generate a unique Digital Business License (LIC-YYYY-XXXXXX) with an annual expiration date upon officer approval of all mandatory documents.
- FR-12: The system shall generate a dynamic QR code for each issued license pointing to the public verification endpoint (verify.php).
- FR-13: The system shall provide a public verification page accessible to any user or mobile device to inspect license authenticity and status (VALID, EXPIRED, SUSPENDED).
- FR-14: The system shall enable vendors to request license renewals and permit officers to review and grant one-year validity extensions.

3.2.3 E-Commerce Marketplace & Order Management:

- FR-15: The system shall allow approved vendors to manage their product catalog (Create, Read, Update, Delete) including title, slug, description, price, stock quantity, primary image, and status.
- FR-16: The system shall enable consumers to browse verified products with category filtering, keyword search, price sorting, and vendor store filtering.

- FR-17: The system shall maintain persistent customer shopping carts tied to authenticated user sessions.
- FR-18: The system shall execute atomic database transactions for checkout, validating stock availability, recalculating item totals on the backend, deducting inventory quantities, generating a unique order number (ORD-YYYY-XXXXXX), and clearing the cart.
- FR-19: The system shall support Cash on Delivery (COD) and a simulated digital gateway portal for eSewa, Khalti, and ConnectIPS.
- FR-20: The system shall maintain customer order tracking displaying status progressions: 'Pending Payment', 'Paid', 'Processing', 'Shipped', 'Delivered', 'Cancelled', and 'Refunded'.
- FR-21: The system shall permit vendors to update the fulfillment status of orders containing items from their catalog.
- FR-22: The system shall automatically restore product inventory levels upon order cancellation.
- FR-23: The system shall restrict product ratings and reviews (1 to 5 stars) strictly to customers who have an authenticated order with 'Delivered' status for that product.

3.2.4 Civic Services & Grievance Redressal (E-Governance):

- FR-24: The system shall display a municipal government services directory detailing departmental ownership, processing times, fee structures, and requirements.
- FR-25: The system shall allow citizens to apply digitally for municipal services (SRV-YYYY-XXXXXX) and attach supporting documentation.
- FR-26: The system shall allow citizens to lodge public grievances (CMP-YYYY-XXXXXX) with category selection (Vendor, Product, Delivery, Municipal Service), priority rating, written description, photographic/document evidence, and Leaflet GIS map pinning.
- FR-27: The system shall allow officers and administrators to view complaints on an interactive GIS map, assign complaints to departments, investigate issues, and record official resolutions.

3.2.5 Platform Administration & Security Auditing:

- FR-28: The system shall provide an Administrator Dashboard featuring real-time database metric counters and dynamic Chart.js visualizations (Vendors by Status, Products by Category, Complaints by Category).
- FR-29: The system shall log significant administrative, financial, and security actions into an immutable audit trail (audit_logs) capturing user ID, action name, target entity, entity ID, description, and client IP address.
- FR-30: The system shall provide instant CSV report generation for vendors, orders, and complaints.

3.3 Non-Functional Requirements

Non-functional requirements specify qualitative benchmarks, security constraints, and operational characteristics:

1. Security:

- Password encryption using PHP's native BCrypt algorithm with standard work factor.
- 100% parameterization of SQL statements via PDO prepared queries to eliminate SQL injection vulnerabilities.
- Synchronizer Token Pattern CSRF defense on all state-altering POST requests.
- Multipurpose Internet Mail Extensions (MIME) validation and file extension whitelisting on file uploads.
- Comprehensive XSS defense through automated HTML entity encoding (htmlspecialchars) and input sanitization.
- Secure session handling with HttpOnly and SameSite cookie attributes.

2. Performance & Responsiveness:

- Sub-second server response times (< 500ms) for standard catalog browsing and dashboard rendering under typical local network loads.
- Database queries optimized with foreign key indexes and limit clauses.
- Asynchronous AJAX handling for cart addition and interactive map rendering without page reloads.

3. Usability & User Experience:

- Modern glassmorphic visual aesthetic with coherent typography, consistent spacing, and intuitive iconography (Font Awesome 6).

- Fully responsive layout adapting dynamically to mobile smartphones, tablets, laptops, and desktop displays via Bootstrap 5 flexbox and grid systems.
- Instant client-side bilingual language switching (English / Nepali) without requiring page refresh.

4. Reliability & Data Integrity:

- Atomic database transactions (BEGIN TRANSACTION, COMMIT, ROLLBACK) for multi-table updates during order placement and payment execution.
- Foreign key cascading rules and strict data type definitions preventing orphaned records.

5. Maintainability & Modularity:

- Separation of concerns across configuration (config/), business middleware (includes/), backend API endpoints (api/), role modules (admin/, officer/, vendor/), and view templates.

3.4 Hardware Requirements

The hardware requirements for hosting and operating SmartGov Market are detailed below:

Development / Server Environment:

- Processor: Intel Core i3 / AMD Ryzen 3 or higher (2.0 GHz multi-core)
- Memory (RAM): Minimum 4 GB (8 GB recommended for concurrent database testing)
- Storage: Minimum 500 MB free disk space for application files, uploads, and database tables
- Network: Standard Network Interface Card (NIC) with TCP/IP protocol support

Client / User Environment:

- Device: Any standard desktop PC, laptop, tablet, or smartphone
- Display Resolution: Minimum 360x640 (mobile) up to 1920x1080 (Full HD desktop)
- Input: Touch screen, mouse, or keyboard

3.5 Software Requirements

The software prerequisites for development, hosting, and client execution:

Server & Hosting Stack:

- Operating System: Microsoft Windows 10/11, Linux (Ubuntu/Debian/CentOS), or macOS
- Web Server: Apache HTTP Server 2.4+ (configured via XAMPP)
- Backend Language: PHP 8.0 or higher (PHP 8.2+ recommended with PDO, pdo_mysql, and fileinfo extensions enabled)
- Database Engine: MySQL 8.0+ or MariaDB 10.4+ with InnoDB engine and utf8mb4 character set

Client Stack:

- Web Browser: Google Chrome (v90+), Mozilla Firefox (v88+), Microsoft Edge (v90+), or Apple Safari (v14+)
- JavaScript: Client-side JavaScript execution enabled (for Leaflet GIS, Chart.js, and i18n switcher)

3.6 User Characteristics

The platform addresses four user personas with diverse technical proficiencies:

- Citizens / Customers: General public possessing basic digital literacy; require simple search, intuitive shopping cart navigation, and transparent complaint submission.
- Vendors / Merchants: Local business owners possessing moderate computer literacy; require straightforward product addition forms, document uploaders, and clear licensing renewal notifications.
- Government Inspection Officers: Municipal staff with administrative computer experience; require clear document inspection queues, checklist verification tools, and remark fields.
- System Administrators: Advanced technical personnel requiring access to user management, database audit logs, system configurations, and CSV analytical exports.

3.7 System Constraints

The platform is subject to several design and deployment constraints:

- Must run efficiently within standard LAMP/WAMP local hosting environments (e.g. XAMPP) without requiring node-based runtime daemons.
- Must not depend on proprietary third-party paid software licenses.
- Payment gateway interactions must be modeled as clean simulated gateways for academic evaluation, preventing unintended financial charges during grading and demonstration.
- File uploads must be constrained to local disk directories (uploads/) with safe hex naming conventions.

3.8 Feasibility Study

A four-dimensional feasibility study was conducted prior to system development:

1. **Technical Feasibility:** The chosen technology stack—PHP 8.2, MariaDB, Bootstrap 5, Leaflet.js, and Chart.js—is mature, battle-tested, extensively documented, and supported by a vast global developer ecosystem. All required features (file uploads, geospatial mapping, transactional ordering, QR rendering) are fully achievable using these open-source tools. Technical feasibility is rated HIGH.
2. **Economic Feasibility:** The entire software stack is open source and freely redistributable under GPL/MIT licenses. Development was executed using existing personal computing equipment without incurring software licensing fees, domain expenses, or proprietary server costs. Economic feasibility is rated HIGH.
3. **Operational Feasibility:** The platform mirrors established municipal governance structures (Departments, Officers, Licenses) and popular commercial checkout workflows. Minimal training is required for municipal officers and vendors. Operational feasibility is rated HIGH.
4. **Schedule Feasibility:** Development was planned across structured sprint cycles (Database Schema Design, Authentication & RBAC, Licensing Module, E-Commerce Core, Civic Grievance & GIS, Security & Testing) and completed within the allocated academic semester schedule. Schedule feasibility is rated HIGH.

CHAPTER 4: SYSTEM DESIGN

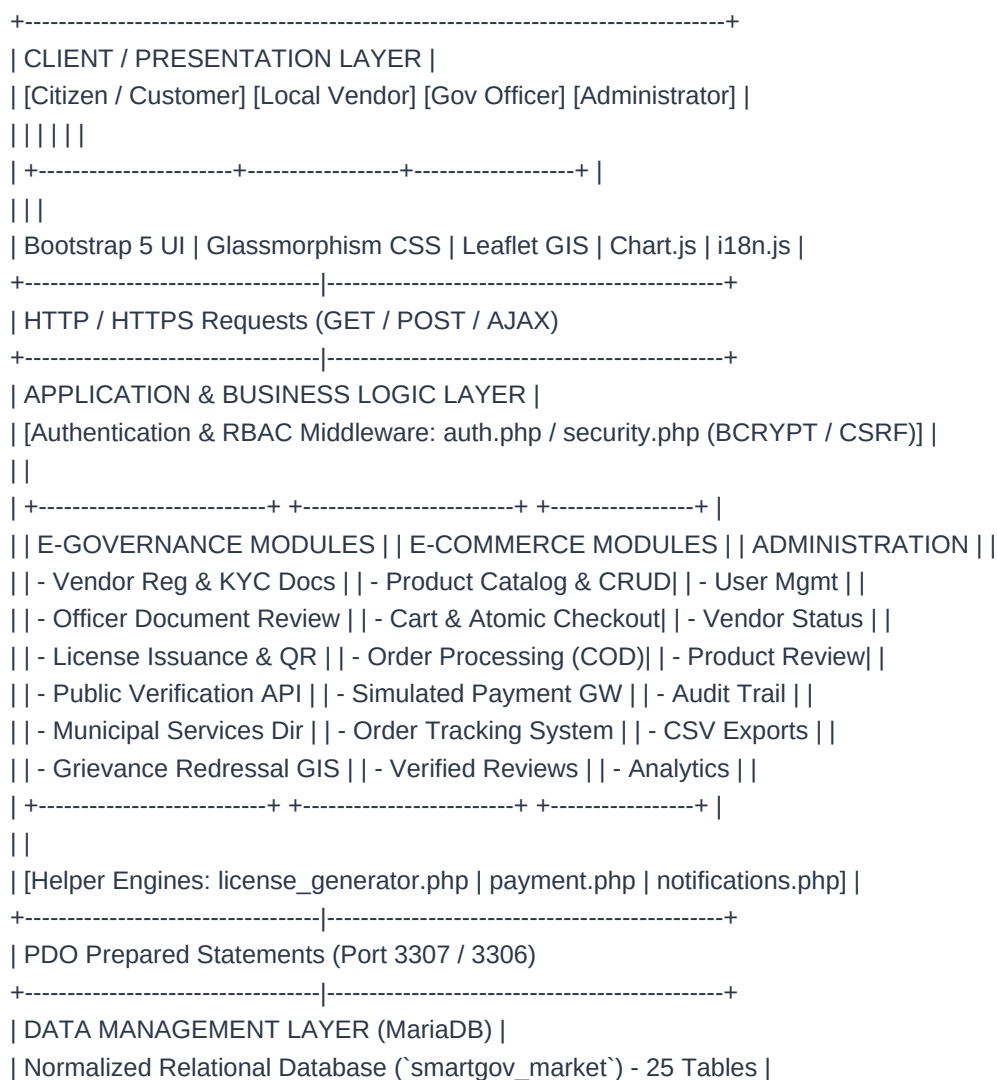
4.1 System Architecture

SmartGov Market employs a 3-Tier Layered Web Architecture optimized for security, performance, and procedural-modular clarity in PHP. The three tiers are organized as follows:

1. Presentation Tier (Client Layer): Executed within the client's web browser. It renders HTML5 semantic markup, Bootstrap 5 responsive styling, custom glassmorphic CSS, Font Awesome 6 icons, interactive Leaflet.js maps, dynamic Chart.js canvases, and client-side JavaScript for bilingual localization (i18n.js) and DOM manipulation.
2. Application / Business Logic Tier (Server Layer): Executed by the Apache HTTP server running PHP 8.2+. This tier contains core business logic, middleware routing, session validation, Role-Based Access Control (RBAC), cryptographic token generators, file upload processors, atomic transaction controllers, and notification dispatchers.
3. Data Tier (Database Layer): Managed by the MariaDB / MySQL 8.0+ relational database engine. It enforces referential integrity through foreign key constraints, stores normalized tabular data across 25 schema tables, executes atomic transactions (ACID compliance), and maintains system audit logs.

4.2 Overall System Architecture Diagram

The overall architectural interaction between platform users, presentation interfaces, application controllers, and database entities is structured as follows:



| - Users, Roles, Departments, Addresses, Vendors, Vendor Applications, Documents |
 | - Licenses, Gov Services, Service Applications, Categories, Products, Images |
 | - Carts, Cart Items, Orders, Order Items, Payments, Complaints, Reviews, Audits |
 +-----+

4.3 User Roles and Permissions Matrix

The platform strictly enforces Role-Based Access Control (RBAC) across four authenticated user roles and public unauthenticated visitors. The permissions matrix below outlines access rights across major functional modules:

4.4 Use Case Diagram

The functional capabilities exposed to the respective system actors are illustrated in the Use Case model:

SMARTGOV MARKET SYSTEM

```
+-----+
| |
| (Browse Products & Search) <----- [Citizen / Customer] |
| (Manage Shopping Cart) <----- [Citizen / Customer] |
| (Checkout & Select Payment) <----- [Citizen / Customer] |
| (Track Order Status) <----- [Citizen / Customer] |
| (Post Verified Review) <----- [Citizen / Customer] |
| (Apply for Municipal Service) <----- [Citizen / Customer] |
| (Lodge Complaint with GIS Pin) <----- [Citizen / Customer] |
| |
| (Register Vendor Profile) <----- [Local Vendor] |
| (Upload PAN/VAT & Permits) <----- [Local Vendor] |
| (View & Print Digital License) <----- [Local Vendor] |
| (Request License Renewal) <----- [Local Vendor] |
| (Manage Products & Stock) <----- [Local Vendor] |
| (Fulfill Customer Orders) <----- [Local Vendor] |
| |
| (Review Vendor Applications) <----- [Government Officer] |
| (Inspect & Verify Documents) <----- [Government Officer] |
| (Approve & Issue Digital License) <----- [Government Officer] |
| (Investigate Public Grievances) <----- [Government Officer] |
| (Process Municipal Applications) <----- [Government Officer] |
| |
| (Manage All System Users) <----- [System Administrator] |
| (Moderate Vendors & Products) <----- [System Administrator] |
| (View Real-Time Chart Analytics) <----- [System Administrator] |
| (Inspect Security Audit Logs) <----- [System Administrator] |
| (Export Analytical CSV Reports) <----- [System Administrator] |
| |
| (Scan QR & Verify License Status) <----- [Public / Unregistered Visitor] |
| |
+-----+
```

4.5 Use Case Descriptions

Detailed use case specifications for six mission-critical system workflows are presented below:

Use Case UC-01: Vendor Onboarding and Document Submission

- Actor: Local Business Owner (Vendor)
- Pre-conditions: User is registered with basic account credentials.
- Description: The vendor accesses the registration/application portal, submits business details (trading name, business type, PAN/VAT number, physical address, and GIS map coordinates), and uploads required compliance documents (PAN certificate, citizenship copy, municipal trade permit).
- Main Flow:
 1. Vendor navigates to 'vendor/application.php'.
 2. System presents multi-part form and interactive Leaflet map.
 3. Vendor selects physical location on map; system populates latitude and longitude.
 4. Vendor selects compliance files and clicks 'Submit Application'.
 5. System validates file sizes (<5MB) and MIME types (PDF, JPG, PNG).
 6. System saves files to 'uploads/documents/' using randomized hex filenames.
 7. System records application entry (status 'Submitted') and links uploaded documents.
 8. System redirects vendor to application status tracker with confirmation flash message.
- Post-conditions: Application is placed in the Officer Review Queue.

Use Case UC-02: Officer Review and Digital License Issuance

- Actor: Municipal Government Officer
- Pre-conditions: Officer is authenticated with 'officer' or 'admin' role.
- Description: Officer reviews pending vendor applications, inspects attached legal certificates, verifies individual documents, and issues a cryptographically authenticated Digital Business License.
- Main Flow:
 1. Officer navigates to 'officer/vendor-applications.php'.
 2. Officer filters by status 'Submitted' and opens specific application.
 3. System renders application details, vendor metadata, GIS coordinates, and document viewer.
 4. Officer inspects each document; clicks 'Verify Document' (or 'Reject Document' with remarks).
 5. If all documents are verified, Officer clicks 'Approve & Issue License'.
 6. System generates unique serial 'LIC-YYYY-XXXXXX', creates QR code verification link, sets issue and expiration dates (1 year), and marks vendor as 'Verified'.
 7. System dispatches real-time notification to the vendor and logs the action in audit_logs.
- Post-conditions: Digital license is active; vendor is unlocked to publish marketplace products.

Use Case UC-03: Product Purchase and Atomic Checkout

- Actor: Citizen / Customer
- Pre-conditions: Customer is authenticated; items from verified vendors are present in cart.
- Description: Customer reviews cart, inputs shipping address, selects payment method, and places order.
- Main Flow:
 1. Customer navigates to checkout ('checkout.php').
 2. System recalculates item totals, verifies vendor active status, and checks live stock levels.
 3. Customer submits shipping address and chooses payment method (COD or Digital Gateway).
 4. Server initiates an atomic database transaction (beginTransaction).
 5. System generates order number 'ORD-YYYY-XXXXXX' and inserts record into 'orders'.
 6. System transfers cart items into 'order_items' and decrements product inventory.
 7. If COD is selected, system records payment record and sets status to 'Processing'.
 8. System empties customer cart, commits the transaction, sends notifications, and logs audit.
 9. If a digital gateway (eSewa/Khalti/ConnectIPS) was chosen, customer is redirected to 'demo-payment.php'.
- Post-conditions: Order is confirmed; stock is reserved; vendor is alerted for fulfillment.

Use Case UC-04: Public QR Code License Verification

- Actor: Any Citizen, Consumer, or Municipal Inspector (No login required)
- Pre-conditions: User possesses a mobile smartphone with camera or opens direct URL.
- Description: User scans the QR code embedded on a vendor's digital license certificate or inputs the license

number directly into 'verify.php'.

- Main Flow:

1. User accesses 'verify.php?license_no=LIC-YYYY-XXXXXX'.
2. System queries 'licenses' table joined with 'vendors' and 'users'.
3. System evaluates current timestamp against license 'expiry_date'.
4. If expired, system dynamically updates license status to 'EXPIRED'.
5. If valid, system displays green verification badge, business name, owner name, business type, registered address, issue date, and validity expiry.
6. If license number does not exist, system renders red 'INVALID LICENSE' alert.

- Post-conditions: Official licensing validity is confirmed transparently.

Use Case UC-05: Public Grievance / Complaint Lodging

- Actor: Citizen / Customer

- Pre-conditions: User is authenticated.

- Description: Citizen lodges a formal complaint regarding vendor malpractice, defective products, or civic issues with photographic evidence and GIS coordinates.

- Main Flow:

1. Citizen opens 'complaints.php' and clicks 'Lodge New Complaint'.
2. Citizen selects category, priority level, enters description, and specifies address.
3. Citizen interacts with map to pinpoint incident location (populating lat/long).
4. Citizen attaches image/PDF evidence file and submits.
5. System saves complaint record with unique tracking ID 'CMP-YYYY-XXXXXX'.
6. System saves attachment into 'uploads/complaints/' and assigns complaint to relevant department.
7. System logs audit and alerts municipal officers.

- Post-conditions: Complaint appears on officer GIS map and grievance queue.

Use Case UC-06: Verified Purchaser Product Review

- Actor: Customer

- Pre-conditions: Customer must have purchased the product and order status must be 'Delivered'.

- Description: Customer submits star rating (1-5) and written feedback for an authentic purchased item.

- Main Flow:

1. Customer navigates to product details or order history.
2. System executes verification query checking 'orders' and 'order_items' for customer ID, product ID, and status 'Delivered'.
3. If eligible, system renders the review submission form.
4. Customer enters rating (1-5) and qualitative comment; clicks 'Submit Review'.
5. System validates rating bounds and persists record into 'reviews'.
6. System updates product average rating and notifies the vendor.

- Post-conditions: Authentic verified review is displayed on the product page.

4.6 Data Flow Diagrams (DFD)

Data Flow Diagrams illustrate the functional movement of information through the system.

4.6.1 Context-Level DFD (Level 0):

Shows the overall system boundary and interactions with external entities:

[Citizen / Customer]

| ^

Reg Data, Orders, | | Catalog Data, Order Status,
Complaints, Payments v | Notifications, Receipts

+-----+

| |

| SMARTGOV MARKET |



4.6.2 Level 1 DFD - E-Governance Subsystem:

- 1.0 Vendor Registration -> Writes to D1: Vendors & D2: Vendor Applications.
- 2.0 Document Processing -> Writes to D3: Vendor Documents (uploads/documents).
- 3.0 Officer Review & Verification -> Queries D2 & D3; Updates status; Writes to D4: Licenses (Generates LIC-YYYY-XXXXXX and QR Code).
- 4.0 Public Verification -> Queries D4: Licenses & D1: Vendors -> Displays status at verify.php.
- 5.0 Public Complaints -> Citizen inputs -> Writes to D5: Complaints & D6: Attachments -> Officer updates resolution.

4.6.3 Level 1 DFD - E-Commerce Subsystem:

- 1.0 Product Management -> Approved vendor creates catalog items -> Writes to D7: Products & D8: Categories.
- 2.0 Catalog Browsing & Cart -> Citizen selects items -> Writes to D9: Cart & D10: Cart Items.
- 3.0 Checkout & Ordering -> Customer submits delivery address & payment choice -> Transaction reads D7 (stock check), writes to D11: Orders & D12: Order Items, decrements D7 (stock).
- 4.0 Payment Handling -> Reads D11; Writes to D13: Payments (stores TXN-XXXXXXXXX); Updates D11 to Paid/Processing.
- 5.0 Review Submission -> Customer submits review -> Validates against D11 (Delivered check) -> Writes to D14: Reviews.

4.7 Entity Relationship (ER) Diagram

The Entity-Relationship model models 25 normalized database tables in the SmartGov Market relational database. Key entity relationships and cardinalities are summarized below:

- roles (1) ----< (M) users: One role is assigned to many users.
- users (1) ----< (M) addresses: A user can maintain multiple shipping addresses.
- users (1) ---- (1) vendors: A user with role 'vendor' has exactly one business profile.
- vendors (1) ----< (M) vendor_applications: A vendor may submit initial and subsequent renewal applications.
- vendor_applications (1) ----< (M) vendor_documents: An application contains multiple compliance documents.
- vendors (1) ----< (M) licenses: A vendor is issued digital business licenses over time.

- departments (1) ----< (M) government_services: A department administers multiple civic services.
- government_services (1) ----< (M) service_applications: A service receives multiple citizen applications.
- users (1) ----< (M) service_applications: A citizen submits multiple service applications.
- categories (1) ----< (M) products: A category groups multiple products.
- vendors (1) ----< (M) products: A verified vendor lists multiple products.
- products (1) ----< (M) product_images: A product may possess multiple gallery photos.
- users (1) ---- (1) cart: A customer has one active shopping cart.
- cart (1) ----< (M) cart_items: A cart contains multiple itemized lines.
- products (1) ----< (M) cart_items: A product can exist in multiple customer carts.
- users (1) ----< (M) orders: A customer places multiple orders.
- orders (1) ----< (M) order_items: An order contains multiple line items.
- products (1) ----< (M) order_items: A product appears across multiple historical order items.
- vendors (1) ----< (M) order_items: Order items link to the respective selling vendor.
- orders (1) ----< (M) payments: An order has associated payment transaction records.
- users (1) ----< (M) complaints: A citizen lodges multiple complaints.
- departments (1) ----< (M) complaints: Complaints are dispatched to specific departments.
- complaints (1) ----< (M) complaint_attachments: A complaint includes multiple evidence files.
- products (1) ----< (M) reviews: A product accumulates multiple customer reviews.
- users (1) ----< (M) reviews: A user authors multiple reviews.
- orders (1) ----< (M) reviews: A review is anchored to a specific delivered order.
- users (1) ----< (M) notifications: A user receives multiple system notifications.
- users (1) ----< (M) audit_logs: User actions generate multiple immutable audit records.
- users (1) ----< (M) password_resets: A user may generate password recovery tokens.

4.8 Database Design & Data Dictionary

The SmartGov Market relational database consists of 25 normalized tables defined in InnoDB with utf8mb4_unicode_ci encoding. Complete schema structures and field specifications are detailed below:

Table 1: roles

- Purpose: Stores system authorization role definitions for RBAC middleware.
- Columns:
 - * id: INT, Primary Key, Auto Increment. Unique role identifier.
 - * name: VARCHAR(50), Unique, Not Null. Internal role slug ('admin', 'officer', 'vendor', 'customer').
 - * display_name: VARCHAR(100), Not Null. Human-readable role title ('System Administrator').
 - * created_at: TIMESTAMP, Default CURRENT_TIMESTAMP. Creation timestamp.

Table 2: departments

- Purpose: Catalogs municipal governance and inspection departments.
- Columns:
 - * id: INT, Primary Key, Auto Increment. Department ID.
 - * department_name: VARCHAR(150), Unique, Not Null. Name of municipal department.
 - * code: VARCHAR(20), Unique, Not Null. Official departmental acronym ('DCI', 'DAFS', 'CPA').
 - * description: TEXT, Nullable. Functional mandate and responsibilities.
 - * is_active: TINYINT(1), Default 1. Operational status indicator.
 - * created_at: TIMESTAMP, Default CURRENT_TIMESTAMP.

Table 3: users

- Purpose: Master user registry storing credentials and profile information.
- Columns:
 - * id: INT, Primary Key, Auto Increment. User ID.
 - * role_id: INT, Foreign Key referencing roles(id) ON DELETE CASCADE.

- * full_name: VARCHAR(150), Not Null. User's complete legal name.
- * email: VARCHAR(150), Unique, Not Null. Contact and login email address.
- * phone: VARCHAR(30), Not Null. Contact mobile number.
- * password_hash: VARCHAR(255), Not Null. BCRYPT-hashed password string.
- * address: TEXT, Nullable. Street and ward address.
- * municipality: VARCHAR(100), Nullable. Municipal authority.
- * district: VARCHAR(100), Nullable. Administrative district.
- * profile_image: VARCHAR(255), Default 'default-avatar.png'. Profile image path.
- * is_verified: TINYINT(1), Default 0. Account verification flag.
- * is_active: TINYINT(1), Default 1. Account active status.
- * last_login: DATETIME, Nullable. Timestamp of most recent authentication.
- * created_at, updated_at: TIMESTAMP.

Table 4: addresses

- Purpose: Stores primary and alternate delivery addresses for citizens.
- Columns:
 - * id: INT, Primary Key, Auto Increment. Address record ID.
 - * user_id: INT, Foreign Key referencing users(id) ON DELETE CASCADE.
 - * address_line: TEXT, Not Null. Street, house number, and landmark.
 - * municipality: VARCHAR(100), Not Null. Municipality.
 - * district: VARCHAR(100), Not Null. District name.
 - * latitude, longitude: DECIMAL(10,8) / DECIMAL(11,8), Nullable. Geolocation coordinates.
 - * is_default: TINYINT(1), Default 0. Default shipping address flag.
 - * created_at: TIMESTAMP, Default CURRENT_TIMESTAMP.

Table 5: vendors

- Purpose: Stores registered vendor business entities, trade categories, and licensing state.
- Columns:
 - * id: INT, Primary Key, Auto Increment. Vendor ID.
 - * user_id: INT, Unique, Foreign Key referencing users(id) ON DELETE CASCADE.
 - * business_name: VARCHAR(200), Not Null. Registered enterprise trading name.
 - * business_type: VARCHAR(100), Not Null. Enterprise category (e.g. Handicrafts, Agriculture).
 - * pan_vat: VARCHAR(50), Nullable. Permanent Account Number / VAT tax registration.
 - * registration_no: VARCHAR(80), Nullable. Official municipal registration number.
 - * address: TEXT, Not Null. Physical store/workshop location.
 - * municipality: VARCHAR(100), Not Null. Municipality.
 - * district: VARCHAR(100), Not Null. District.
 - * latitude, longitude: DECIMAL(10,8) / DECIMAL(11,8), Defaults (27.7172, 85.3240). Store GIS coordinates.
 - * status: ENUM('Pending','Verified','Approved','Under Review','Correction Required','Rejected','Suspended'), Default 'Pending'.
 - * created_at, updated_at: TIMESTAMP.

Table 6: vendor_applications

- Purpose: Manages the onboarding and inspection review lifecycle of vendor applications.
- Columns:
 - * id: INT, Primary Key, Auto Increment. Application ID.
 - * application_no: VARCHAR(50), Unique, Not Null. Formatted tracking code ('VND-YYYY-XXXXXX').
 - * vendor_id: INT, Foreign Key referencing vendors(id) ON DELETE CASCADE.
 - * department_id: INT, Nullable, Foreign Key referencing departments(id) ON DELETE SET NULL.
 - * assigned_officer_id: INT, Nullable, Foreign Key referencing users(id) ON DELETE SET NULL.
 - * status: ENUM('Draft','Submitted','Under Review','Correction Required','Approved','Rejected','Suspended'), Default 'Submitted'.

- * officer_remarks: TEXT, Nullable. Official review comments and instructions.
- * submitted_at: TIMESTAMP, Default CURRENT_TIMESTAMP.
- * reviewed_at, approved_at: DATETIME, Nullable.

Table 7: vendor_documents

- Purpose: Stores metadata and verification states for uploaded legal compliance documents.
- Columns:
 - * id: INT, Primary Key, Auto Increment. Document ID.
 - * application_id: INT, Foreign Key referencing vendor_applications(id) ON DELETE CASCADE.
 - * document_type: VARCHAR(100), Not Null. Label ('PAN/VAT Certificate', 'Citizenship', 'Municipal Permit').
 - * file_name: VARCHAR(255), Not Null. Original client filename.
 - * file_path: VARCHAR(255), Not Null. Storage path ('uploads/documents/hash.ext').
 - * file_size: INT, Default 0. File size in bytes.
 - * verification_status: ENUM('Pending', 'Verified', 'Rejected', 'Resubmission Required'), Default 'Pending'.
 - * officer_remarks: TEXT, Nullable. Document-specific officer audit remarks.
 - * uploaded_at: TIMESTAMP, Default CURRENT_TIMESTAMP.

Table 8: licenses

- Purpose: Official Digital Business Licenses issued to verified vendors.
- Columns:
 - * id: INT, Primary Key, Auto Increment. License record ID.
 - * license_no: VARCHAR(50), Unique, Not Null. Standardized license code ('LIC-YYYY-XXXXXX').
 - * vendor_id: INT, Foreign Key referencing vendors(id) ON DELETE CASCADE.
 - * application_id: INT, Nullable, Foreign Key referencing vendor_applications(id) ON DELETE CASCADE.
 - * issuing_authority: VARCHAR(150), Default 'SmartGov Market Licensing Board'.
 - * issue_date, expiry_date: DATE, Not Null. Effective validity dates.
 - * status: ENUM('VALID', 'EXPIRED', 'SUSPENDED', 'REVOKED'), Default 'VALID'.
 - * renewal_status: ENUM('None', 'Requested', 'Approved', 'Rejected'), Default 'None'.
 - * renewal_requested_at: DATETIME, Nullable.
 - * qr_code_path: VARCHAR(255), Nullable. Verification URL encoded in QR.
 - * created_at: TIMESTAMP, Default CURRENT_TIMESTAMP.

Table 9: government_services

- Purpose: Municipal digital service directory.
- Columns:
 - * id: INT, Primary Key, Auto Increment. Service ID.
 - * department_id: INT, Foreign Key referencing departments(id) ON DELETE CASCADE.
 - * service_name: VARCHAR(200), Not Null. Official service title.
 - * description, requirements: TEXT, Not Null. Service details and document checklist.
 - * processing_time: VARCHAR(100), Not Null. Expected completion SLA (e.g. '2-3 Business Days').
 - * fee: DECIMAL(10,2), Default 0.00. Statutory administrative fee.
 - * status: TINYINT(1), Default 1. Active service flag.
 - * created_at: TIMESTAMP, Default CURRENT_TIMESTAMP.

Table 10: service_applications

- Purpose: Citizen applications submitted for municipal services.
- Columns:
 - * id: INT, Primary Key, Auto Increment. Application ID.
 - * application_no: VARCHAR(50), Unique, Not Null. Tracking code ('SRV-YYYY-XXXXXX').
 - * service_id: INT, Foreign Key referencing government_services(id) ON DELETE CASCADE.
 - * user_id: INT, Foreign Key referencing users(id) ON DELETE CASCADE.
 - * assigned_officer_id: INT, Nullable. Assigned handling officer.

* status: ENUM('Draft','Submitted','Under Review','Processing','Correction Required','Approved','Rejected'), Default 'Submitted'.

* remarks, document_path: TEXT / VARCHAR(255), Nullable. Officer notes and uploaded citizen file.

* submitted_at, updated_at: TIMESTAMP.

Table 11: service_application_documents

- Purpose: Supporting document attachments for citizen service applications.

- Columns: id (PK), application_id (FK), document_type, file_name, file_path, file_size, uploaded_at.

Table 12: categories

- Purpose: Taxonomy for marketplace product classification.

- Columns: id (PK), name (VARCHAR 100, Unique), slug (VARCHAR 100, Unique), description (TEXT), image_path, is_active (TINYINT 1), created_at.

Table 13: products

- Purpose: Merchant catalog items listed on the marketplace.

- Columns:

* id: INT, Primary Key, Auto Increment. Product ID.

* vendor_id: INT, Foreign Key referencing vendors(id) ON DELETE CASCADE.

* category_id: INT, Foreign Key referencing categories(id) ON DELETE CASCADE.

* name: VARCHAR(200), Not Null. Product title.

* slug: VARCHAR(200), Not Null. URL-friendly slug identifier.

* description: TEXT, Not Null. Detailed product specifications.

* price: DECIMAL(10,2), Not Null. Unit retail price in NPR.

* stock_quantity: INT, Not Null, Default 0. Available inventory units.

* image_path: VARCHAR(255), Default 'product-default.jpg'. Primary image.

* status: ENUM('Active','Disabled','Inactive','Out of Stock','Pending Approval'), Default 'Active'.

* created_at, updated_at: TIMESTAMP.

Table 14: product_images

- Purpose: Auxiliary product gallery images.

- Columns: id (PK), product_id (FK), image_path, is_primary (TINYINT 1).

Table 15: cart

- Purpose: Customer active shopping cart header.

- Columns: id (PK), user_id (INT, Unique, FK to users), created_at, updated_at.

Table 16: cart_items

- Purpose: Itemized lines inside active shopping carts.

- Columns: id (PK), cart_id (FK), product_id (FK), quantity (INT), price (DECIMAL 10,2).

Table 17: orders

- Purpose: Master customer orders.

- Columns:

* id: INT, Primary Key, Auto Increment. Order ID.

* order_no: VARCHAR(50), Unique, Not Null. Order identifier ('ORD-YYYY-XXXXXX').

* customer_id: INT, Foreign Key referencing users(id) ON DELETE CASCADE.

* customer_name, customer_phone: VARCHAR, Nullable. Contact snapshots at checkout.

* total_amount: DECIMAL(10,2), Not Null. Grand total order value.

* shipping_address, municipality, district: TEXT / VARCHAR. Delivery destination.

* payment_method: VARCHAR(50), Default 'Cash on Delivery'.

* payment_status: ENUM('Pending','Paid','Failed','Refunded'), Default 'Pending'.

* status: ENUM('Pending','Pending Payment','Confirmed','Paid','Processing','Shipped','Delivered','Cancelled','Refund Requested','Refunded'), Default

'Processing'.

* created_at, updated_at: TIMESTAMP.

Table 18: order_items

- Purpose: Individual product line items in completed orders.

- Columns: id (PK), order_id (FK), product_id (FK), vendor_id (FK), quantity (INT), price (DECIMAL 10,2), subtotal (DECIMAL 10,2).

Table 19: payments

- Purpose: Payment transaction records and gateway logs.

- Columns:

* id: INT, Primary Key, Auto Increment. Payment record ID.

* order_id: INT, Foreign Key referencing orders(id) ON DELETE CASCADE.

* transaction_id: VARCHAR(100), Unique, Not Null. Transaction code ('TXN-XXXXXXXX' or 'DEMO-XXXXXXXX').

* payment_method: VARCHAR(50), Not Null. Gateway identifier ('Cash on Delivery', 'eSewa', 'Khalti', 'ConnectIPS').

* amount: DECIMAL(10,2), Not Null. Amount captured.

* status: ENUM('Pending','Completed','Failed','Refunded'), Default 'Completed'.

* payment_date: TIMESTAMP, Default CURRENT_TIMESTAMP.

Table 20: complaints

- Purpose: Public grievances and civic complaints.

- Columns:

* id: INT, Primary Key, Auto Increment. Complaint ID.

* complaint_no: VARCHAR(50), Unique, Not Null. Grievance tracking code ('CMP-YYYY-XXXXXX').

* user_id: INT, Foreign Key referencing users(id) ON DELETE CASCADE.

* category: ENUM('Vendor','Product','Order','Payment','Delivery','Government Service','License','Other'), Not Null.

* priority: ENUM('Low','Medium','High','Urgent'), Default 'Medium'.

* description, location_address: TEXT. Written statement and location.

* latitude, longitude: DECIMAL(10,8) / DECIMAL(11,8). Incident GIS coordinates.

* department_id: INT, Nullable, Foreign Key referencing departments(id) ON DELETE SET NULL.

* assigned_officer_id: INT, Nullable, Foreign Key referencing users(id) ON DELETE SET NULL.

* status: ENUM('Submitted','Under Review','Assigned','Under Investigation','In Progress','Resolved','Rejected','Closed'), Default 'Submitted'.

* resolution: TEXT, Nullable. Official findings and corrective action.

* created_at, updated_at: TIMESTAMP.

Table 21: complaint_attachments

- Purpose: Photographic or documentary evidence files for complaints.

- Columns: id (PK), complaint_id (FK), file_path, file_name.

Table 22: reviews

- Purpose: Verified purchaser product ratings and feedback.

- Columns: id (PK), product_id (FK), user_id (FK), order_id (FK), rating (INT CHECK 1-5), comment (TEXT), created_at.

Table 23: notifications

- Purpose: Real-time user dashboard notification messages.

- Columns: id (PK), user_id (FK), title, message, link, is_read (TINYINT 1, Default 0), created_at.

Table 24: audit_logs

- Purpose: Immutable security audit trail.

- Columns: id (PK), user_id (FK, Nullable), action, entity, entity_id, description, ip_address, created_at.

Table 25: password_resets

- Purpose: Time-sensitive cryptographic tokens for password recovery.
- Columns: id (PK), user_id (FK), token (VARCHAR 64, Unique), expires_at (DATETIME), used (TINYINT 1), created_at.

4.9 Class and Module Design

The SmartGov Market application architecture is organized into functional helper modules and controllers:

1. Database Connection Controller (config/database.php):

- getConnection(): Singleton-pattern function returning a persistent PDO connection. Implements multi-port fallback logic (attempts port 3307, falls back to 3306) to ensure resilience across custom XAMPP configurations.
- ensureSchemaUpgrades(): Dynamic schema migration runner that automatically verifies and provisions auxiliary tables (password_resets, service_application_documents) and synchronizes ENUM column expansions.

2. Authentication & Authorization Middleware (includes/auth.php):

- enforceSessionTimeout(): Enforces 7200-second session inactivity limit.
- isLoggedIn(), currentUser(), currentRole(), hasRole(\$roles): Contextual authentication helpers.
- requireLogin(), requireRole(\$roles): Route protection interceptors redirecting unauthorized access attempts.
- loginUser(\$user), logoutUser(): Secure session initializers and destroyers.

3. Security & Validation Engine (includes/security.php):

- csrf_token(), csrf_field(), verifyCsrf(), requireCsrf(): Synchronizer token generator and validator.
- storeUploadedFile(): Secure file handler validating MIME types via finfo, checking extensions against whitelists, and storing files under 16-byte random hex names.
- restoreOrderStock(): Restores reserved inventory on cancelled orders.

4. Licensing & QR Generation Engine (includes/license_generator.php):

- issueDigitalLicense(): Orchestrates digital license creation, generates sequential license serials, sets validity dates, and constructs QR verification URIs.
- getVendorLicense(), verifyLicenseByNo(): License retrieval and real-time expiration validation.
- requestLicenseRenewal(), approveLicenseRenewal(): License lifecycle management.

5. Payment Processing Engine (includes/payment.php):

- processOrderPayment(): Atomic transaction handler for Cash on Delivery and payment gateway confirmations, creating payment rows and alerting customers.

6. Notifications Engine (includes/notifications.php):

- sendNotification(): Inserts user notification alerts and handles unread counter increments.

4.10 Sequence Diagrams

Sequence diagrams model the chronological exchange of messages between actors and system components for vital workflows:

4.10.1 Citizen Registration Sequence:

Citizen -> Browser -> register.php -> auth.php -> Database (users)

1. Citizen fills registration form and clicks 'Register'.
2. Browser sends POST request with CSRF token to register.php.
3. register.php validates inputs (email uniqueness, password length).
4. register.php hashes password using password_hash(BCRYPT).
5. Database inserts new user record; returns user_id.
6. auth.php initializes session via loginUser(); logs audit in audit_logs.
7. System redirects citizen to index.php with welcome flash message.

4.10.2 Vendor Onboarding & Officer License Approval Sequence:

Vendor -> vendor/application.php -> Officer -> officer/application-details.php -> license_generator.php ->

Database

1. Vendor submits application form and compliance files.
2. Server validates files via `storeUploadedFile()`, writes to `vendor_applications` and `vendor_documents`.
3. Application enters Officer queue with status 'Submitted'.
4. Officer logs in and navigates to application review screen.
5. Officer inspects documents via document viewer; marks files as 'Verified'.
6. Officer clicks 'Approve & Issue License'.
7. `license_generator.php` generates 'LIC-YYYY-XXXXXX', computes 1-year expiry, updates vendor status to 'Verified'.
8. Server sends notification to Vendor and logs audit trail.

4.10.3 Product Purchase & Atomic Order Sequence:

Customer -> `checkout.php` -> `api/place_order.php` -> Database (`orders`, `order_items`, `products`)

1. Customer clicks 'Place Order' on checkout form.
2. `api/place_order.php` validates CSRF token and cart contents.
3. Server opens database transaction (`beginTransaction`).
4. Server validates stock levels and recalculates totals.
5. Server inserts order into 'orders' ('ORD-YYYY-XXXXXX').
6. Server loops through items: inserts into 'order_items' and decrements 'products.stock_quantity'.
7. If COD: `payment.php` creates payment record; order status set to 'Processing'.
8. Server clears `cart_items`, commits transaction, and notifies vendors.
9. If digital gateway chosen: redirects customer to `demo-payment.php`.

4.11 Activity Diagrams

Activity diagrams illustrate procedural operational workflows:

4.11.1 Vendor Licensing Lifecycle Activity:

[Start] -> Vendor Registers -> Submits Trade Data & Documents -> [Status: Submitted] -> Officer Inspects Documents -> {All Documents Valid?}

- If NO: Officer adds remarks -> [Status: Correction Required] -> Vendor Re-uploads Documents -> [Back to Officer Inspection]

- If Rejected: Officer rejects application -> [Status: Rejected] -> [End]

- If YES: Officer clicks Approve -> System generates Digital License (LIC-YYYY-XXXXXX) -> QR Code Path Created -> Vendor Status set to 'Verified' -> Vendor Permitted to Sell -> [End]

4.11.2 Order & Fulfillment Lifecycle Activity:

[Start] -> Customer Adds Items to Cart -> Navigates to Checkout -> Submits Address -> Selects Payment -> {Payment Method?}

- If COD: System confirms Order -> [Status: Processing] -> Vendor Prepares Goods -> Vendor Marks as Shipped -> Customer Receives Goods -> Vendor Marks as Delivered -> Customer Submits Verified Review -> [End]

- If Digital Gateway: Customer redirected to `demo-payment.php` -> Customer enters demo ID -> Demo Payment Processed -> [Status: Paid] -> Vendor Fulfills Order -> [Status: Delivered] -> [End]

Table 4.1: Role-Based Access Control (RBAC) Permissions Matrix

Capability	Guest	Citizen	Vendor	Officer	Admin
Browse Marketplace Catalog	Full	Full	Full	Full	Full
Public QR Verification	Full	Full	Full	Full	Full
Account Registration	Yes	Self	Self	Admin	Master
Manage Shopping Cart	No	Full	Customer Mode	No	Testing
Place Orders & Checkout	No	Full	Customer Mode	No	Testing
Track Order Delivery	No	Own	Store Items	No	All
Verified Product Review	No	Delivered	No	No	Audit
Lodge Public Grievance	No	GIS Map	GIS Map	GIS Map	GIS Map
Municipal Service Apps	No	Full	Full	No	All
Upload Legal Documents	No	No	Own Store	No	Override
Print Digital License	No	No	Own Lic	All	All
Product Catalog CRUD	No	No	Verified	No	Moderate
Fulfill Customer Orders	No	No	Own Items	No	Override
Inspect Vendor Queue	No	No	No	Full	Full
Verify / Reject Files	No	No	No	Full	Full
Approve & Issue License	No	No	No	Full	Full
Resolve Civic Grievances	No	No	No	Assigned	All
Real-Time Chart.js	No	No	Sales	KPI	Master
Audit Logs & CSV Export	No	No	No	CSV	Full

CHAPTER 5: SYSTEM IMPLEMENTATION

5.1 Development Environment

The SmartGov Market platform was engineered and validated within a standardized modern web development environment:

- Operating System: Microsoft Windows 11 Professional (64-bit)
- Local Server Suite: XAMPP Version 8.2 (incorporating Apache 2.4.58 and MariaDB 10.4.32)
- Database Ports: MariaDB configured on custom service port 3307 with transparent fallback to standard port 3306.
- Integrated Development Environment (IDE): Visual Studio Code equipped with PHP Intelephense, SQLTools, and Git version control integration.
- Web Client Testing: Google Chrome 124, Mozilla Firefox 125, Microsoft Edge 124.

5.2 Technologies Used

The platform was constructed using a cohesive modern technology stack without extraneous bloat:

1. Backend: PHP 8.2+ utilizing object-oriented PDO extensions, BCrypt password hashing, session hardening, and JSON serialization.
2. Relational Database: MariaDB / MySQL with InnoDB storage engine, foreign key cascade constraints, transactions, and utf8mb4 collation.
3. Frontend Framework: HTML5 semantic tags, Bootstrap 5.3 CSS grid and components, supplemented by custom glassmorphic CSS styling.
4. Typography & Iconography: Inter Google Font family and Font Awesome 6.5 Free Vector Icons.
5. Geospatial & Mapping: Leaflet.js (v1.9.4) paired with OpenStreetMap tile servers for interactive mapping, geocoding pin-drops, and store coordinates.
6. Data Visualization: Chart.js (v4.4.1) for dynamic database-driven admin analytics (pie, bar, and donut charts).
7. QR Code Engine: QRCode.js client-side generator with dynamic fallback to QRServer REST API.
8. Localization: Client-side internationalization dictionary (i18n.js) providing English and Nepali (■■■■■■■) language toggle.

5.3 Project Folder Structure

The repository is structured logically to enforce clean separation between configuration, libraries, role portals, uploaded media, and public web endpoints:

SmartGov-Market/

- assets/
- ■■■■ css/ # Custom glassmorphic stylesheets & Bootstrap bundles
- ■■■■ images/ # Brand graphics, SVG placeholders, and system logos
- ■■■■ js/ # i18n localization, Leaflet map helpers, Chart.js loaders
- config/
- ■■■■ app.php # Core application constants, sanitization, flash messages, audit helper
- ■■■■ database.php # Multi-port PDO connection singleton & auto-schema migration
- database/
- ■■■■ schema.sql # DDL schema definition for 23 base tables
- ■■■■ update_schema.sql # Incremental schema migration scripts
- ■■■■ seed.php # Demo dataset populator for users, vendors, products, complaints
- ■■■■ init_db.php # Automated CLI database installation runner
- includes/
- ■■■■ auth.php # Session management and RBAC authorization middleware
- ■■■■ header.php # Global HTML header, navigation, and flash message renderer

- ■■■■ footer.php # Global HTML footer, JavaScript bundle, and copyright notice
- ■■■■ navbar.php # Top navigation bar with bilingual language switcher
- ■■■■ sidebar.php # Dynamic role-based dashboard navigation sidebars
- ■■■■ license_generator.php # Digital license creation and QR code encoder
- ■■■■ payment.php # COD and simulated payment processing controller
- ■■■■ security.php # CSRF tokens, secure file uploads, stock restoration
- ■■■■ notifications.php # Dashboard notifications engine
- uploads/
- ■■■■ documents/ # Vendor legal compliance documents (PAN, ID, permits)
- ■■■■ products/ # Product catalog images uploaded by merchants
- ■■■■ complaints/ # Photo and document attachments supporting complaints
- ■■■■ profiles/ # User profile avatars
- admin/ # Administrator management console (13 controller files)
- ■■■■ dashboard.php, users.php, vendors.php, products.php, categories.php,
- ■■■■ orders.php, complaints.php, licenses.php, departments.php, services.php,
- ■■■■ reports.php, settings.php, audit-logs.php
- officer/ # Government officer inspection portal (6 controller files)
- ■■■■ dashboard.php, vendor-applications.php, application-details.php,
- ■■■■ complaints.php, licenses.php, reports.php
- vendor/ # Merchant portal (10 controller files)
- ■■■■ dashboard.php, register.php, application.php, license.php,
- ■■■■ products.php, add-product.php, edit-product.php, orders.php, reviews.php, sales.php
- api/ # Asynchronous backend endpoints
- ■■■■ cart.php # Cart addition, quantity update, and item removal
- ■■■■ place_order.php # Atomic order placement and transaction controller
- index.php # Marketplace landing page with featured verified products
- login.php / register.php / logout.php / forgot-password.php / reset-password.php
- products.php / product-details.php / cart.php / checkout.php / demo-payment.php
- orders.php / order-details.php / transactions.php / reviews.php
- complaints.php / complaint-details.php / map-view.php
- government-services.php / service-details.php / applications.php
- verify.php # Public QR digital business license verification page

5.4 Database Connection & Dynamic Resilience

The database layer is managed through config/database.php via the getDBConnection() function. Key technical implementation details include:

1. Multi-Port Fallback: In local environments, MySQL/MariaDB frequently runs on port 3307 due to port conflicts with existing MySQL installations on 3306. The connection logic loops through ports [3307, 3306], successfully binding to the active service port transparently.
2. PDO Configuration: Configured with PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION for robust error handling, PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC for performant associative arrays, and PDO::ATTR_EMULATE_PREPARES => false to enforce real prepared statements on the database server.
3. Automated Schema Upgrades: The ensureSchemaUpgrades() function automatically checks for the existence of password_resets and service_application_documents tables upon initialization, verifies supplementary columns (pan_vat, registration_no, customer_name), and updates ENUM column constraints safely.

5.5 Authentication, Session Management & RBAC

User authentication and authorization are encapsulated within includes/auth.php:

- Password Security: Handled via PHP's native password_hash(\$password, PASSWORD_BCRYPT) and verified via password_verify(). Plaintext passwords are never stored or logged.
- Session Hardening: Upon successful credential verification, loginUser() executes session_regenerate_id(true) to mitigate session fixation attacks, records the user's last login timestamp, and initializes role session variables.
- Inactivity Timeout: enforceSessionTimeout() computes elapsed time since last_activity; if it exceeds 7,200 seconds (2 hours), the user is automatically logged out and redirected to login with an alert.
- Route Protection: requireRole(\$roles) intercepts unauthorized requests. If an authenticated vendor attempts to access admin/dashboard.php, they are intercepted and redirected to vendor/dashboard.php with an 'Unauthorized Access' flash warning.

5.6 User / Citizen Module Implementation

The Citizen subsystem powers civic and commercial interaction:

- Profile & Address Book: profile.php allows citizens to manage contact information, update passwords, and maintain primary delivery addresses.
- Digital Applications: applications.php displays historical municipal service requests submitted by the citizen, complete with downloadable submitted files, officer review remarks, and live approval badges.
- Orders & Transactions: orders.php and order-details.php provide a visual delivery progress timeline (Pending Payment -> Paid -> Processing -> Shipped -> Delivered). transactions.php displays all historical financial transactions with transaction IDs, payment methods, and timestamps.
- Grievance Redressal: complaints.php allows citizens to lodge complaints with category selection, priority, GIS map pinpointing, and photo evidence uploads.

5.7 Vendor Module Implementation

The Vendor subsystem governs commercial participation and compliance:

- Multi-Step Onboarding: vendor/application.php allows business owners to input enterprise information, trade sectors, and drag a marker on a Leaflet GIS map to pin their store coordinates.
- Compliance Document Upload: Implemented using storeUploadedFile(), validating file MIME types and storing files with randomized hex names under uploads/documents/.
- Digital License Certificate: vendor/license.php renders the official digital business license certificate, issue/expiry dates, issuing municipal authority, and live QR code. It features a one-click 'Print Official License' button with dedicated print CSS media queries.
- Catalog & Inventory Management: vendor/products.php, add-product.php, and edit-product.php provide complete CRUD capabilities over products, real-time stock updating, and image uploads.
- Order Fulfillment: vendor/orders.php alerts vendors when customer orders contain items from their inventory, enabling them to progress fulfillment states to 'Shipped' or 'Delivered'.

5.8 E-Commerce Marketplace Implementation

The marketplace subsystem bridges consumers and verified merchants:

- Verified-Only Product Catalog: products.php executes dynamic SQL joins filtering products by status = 'Active' and vendors status IN ('Approved', 'Verified'). Products from unapproved vendors are completely excluded from public browsing.
- Shopping Cart: cart.php and api/cart.php maintain customer shopping carts with live price lookups, stock checks, and asynchronous AJAX quantity adjustments.
- Atomic Checkout Transaction: api/place_order.php executes the entire checkout within a single database transaction:
 - * Backend price validation: Recalculates total amount directly from product table prices, thwarting client-side price tampering.
 - * Stock reservation: Decrements stock_quantity atomically.
 - * Multi-table insertion: Inserts master order, line items, and payment rows.
 - * Notification dispatch: Sends instant in-app alerts to both the buyer and all selling vendors.
- Verified Reviews: reviews.php enforces that product reviews can only be submitted by customers who have an existing order record for that product with status = 'Delivered'.

5.9 E-Governance Module Implementation

The e-governance subsystem modernizes municipal services and regulatory enforcement:

- Municipal Services Catalog: government-services.php and service-details.php showcase municipal services, processing times, fee schedules, and required documentation.
- Officer Inspection Queue: officer/vendor-applications.php presents inspection officers with pending applications, filtering options, and applicant summaries.
- Granular Document Viewer & Audit: officer/application-details.php renders applicant legal documents, allowing officers to individually verify or reject each document with custom remarks.
- Automated Digital License Generation: Upon officer approval, license_generator.php automatically issues an official license (LIC-YYYY-XXXXXX), computes an annual expiration date, marks the vendor as 'Verified', and encodes the public verification URL into a QR code.
- Public QR Verification Endpoint: verify.php provides a lightweight, public interface accessible by scanning the license QR code. It dynamically validates whether the license is VALID, EXPIRED, or SUSPENDED.
- GIS Public Grievance Resolution: officer/complaints.php allows officers to inspect lodged complaints, examine uploaded evidence photos, review incident GIS map coordinates, and record formal resolution findings.

5.10 Administrator Module Implementation

The Administrator console (admin/) provides centralized oversight across the entire platform:

- Real-Time Analytical Dashboard: admin/dashboard.php queries live database counts for total users, approved vs. pending vendors, total orders, and sales revenue. It dynamically initializes three Chart.js charts: Vendors by Status (Doughnut), Products by Category (Bar), and Grievances by Category (Doughnut).
- Merchant Moderation: admin/vendors.php enables administrators to inspect, approve, reject, or suspend vendor operating privileges.
- Product Oversight: admin/products.php allows administrators to toggle product visibility or disable non-compliant items.
- Security Audit Log Viewer: admin/audit-logs.php provides a searchable table of all recorded audit events (timestamp, action, entity, user ID, client IP).
- CSV Export Engine: admin/reports.php generates instant, downloadable CSV reports for vendors, marketplace sales orders, and civic complaints using PHP's native fputcsv() streaming.

5.11 Payment System Implementation (Demo Gateway)

The payment subsystem supports dual payment paradigms:

1. Native Cash on Delivery (COD): Automatically records payment status as 'Pending' and order status as 'Processing'. The customer pays upon physical goods delivery.
2. Simulated Digital Payment Gateway (eSewa / Khalti / ConnectIPS): Implemented via demo-payment.php and includes/payment.php. Designed strictly as an academic and demonstration gateway:
 - Captures order total and displays branded digital wallet interfaces (eSewa green, Khalti purple, ConnectIPS navy).
 - Prompts the user for a demo mobile/account number (e.g. 9841000000).
 - Generates a unique transaction identifier ('DEMO-XXXXXXX' or 'TXN-XXXXXXX').
 - Atomically transitions the order to payment_status = 'Paid' and status = 'Paid'.
 - Explicitly informs users that no real banking credentials or live financial charges are involved.

5.12 Security Implementation

SmartGov Market embeds defensive security controls across all architectural layers:

- SQL Injection Defense: 100% of database interactions utilize PDO prepared statements with bound parameters. Zero dynamic SQL string concatenation is permitted.
- Cross-Site Scripting (XSS) Prevention: All dynamic outputs rendered into HTML contexts pass through the sanitize() helper, which executes htmlspecialchars(..., ENT_QUOTES, 'UTF-8').
- Cross-Site Request Forgery (CSRF) Defense: Forms include hidden CSRF tokens generated via random_bytes(32). The requireCsrf() middleware validates tokens on all POST requests using constant-time comparison (hash_equals).
- Secure File Upload Handling: storeUploadedFile() enforces a 5MB maximum file size, inspects real file content MIME types using finfo(FILEINFO_MIME_TYPE), validates against strict extensions (pdf, jpg, jpeg, png), and stores files with randomized 16-byte hex filenames to prevent directory traversal and executable script execution.
- Password Security: User passwords are encrypted with BCrypt using PHP's native password_hash().
- Session Protection: Enforces HttpOnly and SameSite=Lax cookie policies, session ID regeneration, and a 2-hour inactivity timeout.

5.13 UI/UX Design and Bilingual Localization

The user interface is designed around a modern glassmorphic theme:

- Visual Hierarchy: Clean card-based layouts with subtle drop-shadows (box-shadow: 0 4px 20px rgba(0,0,0,0.06)), rounded borders (border-radius: 12px), and a curated civic color palette (Navy Blue #1a2b4c, Government Accent Red #dc3545, Emerald Green #198754).
- Dynamic Animations: Micro-interactions and hover effects enhance user engagement without degrading browser performance.
- Bilingual Localization (i18n): Implemented via assets/js/i18n.js. Clicking the language toggle switch dynamically translates interface elements (navigation links, buttons, table headers) between English and Nepali (नेपाली) in the client DOM without requiring server reload.

CHAPTER 6: USER INTERFACE AND SCREENSHOTS

This chapter documents the primary user interfaces implemented within the SmartGov Market platform. Each screen is detailed with its operational purpose, permitted user roles, and visual presentation layout.

■ [INSERT SCREENSHOT: MARKETPLACE HOME PAGE (index.php) HERE]

Route: index.php | **Authorized Roles:** Public (All Users, Unauthenticated Visitors, Citizens)

Figure 6.1: Marketplace Home Page (Landing Portal)

Description: The landing page presents the primary entry point to SmartGov Market. It features a hero banner highlighting municipal e-governance and verified local commerce, quick search and category filters, featured verified products, active merchant counts, and links to public municipal services and QR license verification.

■ [INSERT SCREENSHOT: USER LOGIN SCREEN (login.php) HERE]

Route: login.php | **Authorized Roles:** Public / Unauthenticated Users

Figure 6.2: User Authentication & Sign In Screen

Description: Provides secure authentication with email and password inputs, role-based automatic dashboard redirection (admin, officer, vendor, customer), credential validation against BCrypt hashes, and links to self-service password recovery and citizen/vendor registration.

■ [INSERT SCREENSHOT: CITIZEN REGISTRATION SCREEN (register.php) HERE]

Route: register.php | **Authorized Roles:** Public / New Citizens

Figure 6.3: Citizen / Customer Account Registration

Description: Allows citizens to create an account by entering full name, email, contact telephone, physical street address, municipality, and administrative district. Enforces CSRF token validation and password confirmation.

■ [INSERT SCREENSHOT: VENDOR APPLICATION & DOCUMENT UPLOAD (vendor/application.php) HERE]

Route: vendor/register.php & vendor/application.php | **Authorized Roles:** Registered Business Owners (Role: Vendor)

Figure 6.4: Vendor Multi-Step Onboarding & Application Portal

Description: Multi-stage onboarding interface where merchants submit business names, trade types, PAN/VAT tax numbers, official registration numbers, drag a Leaflet GIS map marker to pin their store coordinates, and upload legal verification files (PAN/VAT certificates, citizenship IDs, municipal trade permits).

■ [INSERT SCREENSHOT: ADMINISTRATOR DASHBOARD (admin/dashboard.php) HERE]

Route: admin/dashboard.php | **Authorized Roles:** System Administrator (Role: Admin)

Figure 6.5: Administrator Master Control Console

Description: Centralized executive management console displaying live database metric counters (registered users, verified vs. pending vendors, total sales volume, civic complaints) and dynamic Chart.js visualizations (Vendors by Status, Products by Category, Complaints by Category).

■ [INSERT SCREENSHOT: OFFICER VENDOR APPLICATIONS QUEUE ([officer/vendor-applications.php](#)) HERE]

Route: [officer/vendor-applications.php](#) | **Authorized Roles:** Municipal Inspection Officers & Administrators

Figure 6.6: Government Officer Review & Inspection Queue

Description: Tabular inspection inbox displaying submitted vendor applications with application serial numbers (VND-YYYY-XXXXXX), applicant names, trade sectors, submission timestamps, status filters (Submitted, Under Review, Correction Required, Approved), and quick-action review triggers.

■ [INSERT SCREENSHOT: OFFICER APPLICATION REVIEW & APPROVAL ([officer/application-details.php](#)) HERE]

Route: [officer/application-details.php](#) | **Authorized Roles:** Municipal Inspection Officers & Administrators

Figure 6.7: Officer Document Inspection & License Approval Screen

Description: Granular document auditing interface enabling officers to review uploaded certificates, verify or reject individual files with custom remarks, request applicant corrections, and trigger one-click digital license issuance upon full statutory compliance.

■ [INSERT SCREENSHOT: DIGITAL BUSINESS LICENSE CERTIFICATE ([vendor/license.php](#)) HERE]

Route: [vendor/license.php](#) | **Authorized Roles:** Verified Vendors (Role: Vendor)

Figure 6.8: Official Digital Business License Certificate

Description: Renders the official municipal digital business license certificate bearing the standardized code (LIC-YYYY-XXXXXX), business name, owner name, issuing authority, issue date, annual expiration date, an embedded live QR verification code, and a print button.

■ [INSERT SCREENSHOT: PUBLIC QR LICENSE VERIFICATION ([verify.php](#)) HERE]

Route: [verify.php](#) | **Authorized Roles:** Public (Open to any smartphone scanner, consumer, or inspector)

Figure 6.9: Public Digital License QR Verification Endpoint

Description: Lightweight, mobile-responsive public verification page that validates license serial numbers against database records, dynamically checking expiration dates and rendering official status badges (VALID, EXPIRED, SUSPENDED, or INVALID).

■ [INSERT SCREENSHOT: PRODUCT CATALOG & SEARCH ([products.php](#)) HERE]

Route: [products.php](#) | **Authorized Roles:** Public (All Users)

Figure 6.10: Verified Marketplace Catalog & Product Search

Description: Public product browsing portal featuring category sidebar filters, keyword search, price range sorting, stock status badges, and merchant verification badges. Restricts listings exclusively to products from verified, licensed vendors.

■ [INSERT SCREENSHOT: PRODUCT DETAILS & REVIEWS (product-details.php) HERE]

Route: product-details.php | *Authorized Roles:* Public (All Users)

Figure 6.11: Product Details & Verified Customer Reviews Screen

Description: Displays full product specifications, vendor identity and store location, current stock availability, image gallery, 'Add to Cart' controls, and verified-purchaser customer reviews and star ratings.

■ [INSERT SCREENSHOT: SHOPPING CART (cart.php) HERE]

Route: cart.php | *Authorized Roles:* Authenticated Customers (Role: Customer)

Figure 6.12: Interactive Shopping Cart Screen

Description: Displays customer's active shopping cart items, unit prices, subtotal calculations, AJAX-powered quantity adjustment steppers, item removal triggers, and live inventory validation prior to checkout.

■ [INSERT SCREENSHOT: CHECKOUT INTERFACE (checkout.php) HERE]

Route: checkout.php | *Authorized Roles:* Authenticated Customers (Role: Customer)

Figure 6.13: Checkout & Delivery Address Configuration

Description: Streamlined checkout form allowing customers to review order line items, specify delivery addresses (municipality, district, street), and select payment methods (Cash on Delivery, eSewa, Khalti, ConnectIPS).

■ [INSERT SCREENSHOT: SIMULATED PAYMENT GATEWAY (demo-payment.php) HERE]

Route: demo-payment.php | *Authorized Roles:* Authenticated Customers (Role: Customer)

Figure 6.14: Simulated Digital Payment Gateway Portal

Description: Academic demonstration payment gateway rendering branded digital wallet interfaces (eSewa, Khalti, ConnectIPS). Prompts for a demo mobile/account number, processes transactions atomically, and confirms order payments without live banking charges.

■ [INSERT SCREENSHOT: ORDER TRACKING & DETAILS (order-details.php) HERE]

Route: orders.php & order-details.php | *Authorized Roles:* Authenticated Customers (Role: Customer)

Figure 6.15: Customer Order History & Live Delivery Tracking

Description: Order management screen displaying order serials (ORD-YYYY-XXXXXX), date placed, total amount, and a visual five-stage delivery progress tracker (Pending Payment -> Paid -> Processing -> Shipped -> Delivered).

■ [INSERT SCREENSHOT: TRANSACTION HISTORY ([transactions.php](#)) HERE]

Route: [transactions.php](#) | **Authorized Roles:** Authenticated Customers (Role: Customer)

Figure 6.16: Customer Payment Transaction History

Description: Comprehensive financial ledger presenting customer transaction IDs (TXN-XXXXXXXX / DEMO-XXXXXXXX), linked order numbers, captured amounts, payment methods, and gateway transaction timestamps.

■ [INSERT SCREENSHOT: GOVERNMENT SERVICES DIRECTORY ([government-services.php](#)) HERE]

Route: [government-services.php](#) & [service-details.php](#) | **Authorized Roles:** Public / Citizens

Figure 6.17: Municipal Government Services Catalog

Description: Municipal service directory showcasing online government services, administrative fees, estimated SLA processing durations, and document requirement checklists, with direct links to submit digital applications.

■ [INSERT SCREENSHOT: CITIZEN SERVICE APPLICATIONS ([applications.php](#)) HERE]

Route: [applications.php](#) | **Authorized Roles:** Authenticated Citizens (Role: Customer)

Figure 6.18: Citizen Service Applications Tracker

Description: Personal citizen dashboard displaying submitted municipal service applications (SRV-YYYY-XXXXXX), attached files, officer remarks, and live approval statuses.

■ [INSERT SCREENSHOT: PUBLIC GRIEVANCE & GIS MAP ([complaints.php](#)) HERE]

Route: [complaints.php](#) & [map-view.php](#) | **Authorized Roles:** Authenticated Citizens, Officers, and Administrators

Figure 6.19: Public Grievance Lodging & Leaflet GIS Mapping

Description: Citizen grievance portal with form fields for category, priority, description, file evidence upload, and Leaflet.js interactive map for geocoding complaint locations with latitude and longitude pins.

■ [INSERT SCREENSHOT: VENDOR PRODUCT MANAGEMENT ([vendor/products.php](#)) HERE]

Route: [vendor/products.php](#) & [vendor/add-product.php](#) | **Authorized Roles:** Approved Vendors (Role: Vendor)

Figure 6.20: Vendor Store Management & Product CRUD Portal

Description: Vendor catalog management console providing tabular inventory overviews, real-time stock updating, product creation with image uploads, pricing configuration, and active status toggling.

■ [INSERT SCREENSHOT: NOTIFICATIONS CENTER ([notifications.php](#)) HERE]

Route: [notifications.php](#) | **Authorized Roles:** All Authenticated Users

Figure 6.21: Real-Time User Notifications Center

Description: Notification center displaying timestamped system alerts, licensing updates, order status changes, and complaint findings with unread badges and direct navigation links.

■ [INSERT SCREENSHOT: AUDIT LOGS & CSV EXPORT ([admin/reports.php](#)) HERE]

Route: *admin/audit-logs.php* & *admin/reports.php* | **Authorized Roles:** System Administrator (Role: Admin)

Figure 6.22: Security Audit Log & CSV Report Generator

Description: Administrative security audit ledger recording system events, user actions, client IP addresses, and one-click CSV export generators for vendors, marketplace orders, and complaints.

CHAPTER 7: TESTING AND QUALITY ASSURANCE

7.1 Testing Introduction

Software testing constitutes an indispensable phase in the engineering lifecycle to verify that the implemented application conforms to specified functional and non-functional requirements. For SmartGov Market, testing was conducted rigorously across both the E-Governance regulatory pipeline and the E-Commerce transactional engine. The testing suite targeted business logic integrity, data consistency during atomic checkout, role authorization boundaries, and resilience against common web application security vulnerabilities.

7.2 Testing Strategy

A multi-layered testing strategy was adopted:

1. Unit Testing: Individual helper functions, code generators (generateCode), sanitization routines (sanitize), file upload checkers, and currency formatters were tested in isolation.
2. Integration Testing: Inter-module communication was verified, particularly the linkage between officer document approvals and automated license issuance, and the linkage between checkout and stock reduction.
3. System Testing: The end-to-end operational flow was evaluated from citizen registration to order delivery, complaint resolution, and public QR verification.
4. User Acceptance Testing (UAT): Operational scenarios were tested across simulated user roles (Admin, Officer, Vendor, Customer) using predefined demo accounts.

7.3 Unit Testing

Unit tests focused on modular utility functions:

- Code Generator: Tested generateCode('ORD', 'orders', 'order_no'). Confirmed output conforms to 'ORD-YYYY-XXXXXX' and uniqueness is verified via database query.
- Currency Formatter: Tested formatCurrency(1250.5). Confirmed output returns '███ 1,250.50'.
- Input Sanitizer: Tested sanitize('alert(1)'). Confirmed HTML entity encoding returns '<script>alert(1)</script>'.
- Session Inactivity Checker: Tested enforceSessionTimeout() with simulated timestamps. Confirmed automatic logout when elapsed time exceeds 7200 seconds.

7.4 Integration Testing

Integration testing verified database transactions and cross-module events:

- Officer Approval & License Issuance: Confirmed that approving an application in officer/application-details.php atomically updates vendor_applications.status to 'Approved', inserts a row into licenses, updates vendors.status to 'Verified', and creates a notification record.
- Order Placement & Inventory Consistency: Confirmed that checkout in api/place_order.php atomically creates orders, populates order_items, deducts products.stock_quantity, creates payments, and deletes cart_items.
- Cancellation Stock Reversal: Confirmed that restoring an order restores deducted inventory quantities via restoreOrderStock().

7.5 System Testing

System testing evaluated end-to-end workflows across the application lifecycle:

- E2E Scenario 1: New vendor registration -> document upload -> officer audit -> license issuance -> product creation -> marketplace publishing.
- E2E Scenario 2: Customer product discovery -> cart addition -> checkout -> demo gateway payment -> vendor order fulfillment -> customer delivery receipt -> verified review submission.
- E2E Scenario 3: Citizen grievance filing with GIS coordinates -> officer inspection -> departmental resolution.

7.6 User Acceptance Testing (UAT)

UAT evaluated role behavior against the standard demo credentials:

- Administrator (admin@smartgov.gov.np / password123): Verified master console access, user management, audit logs, and CSV export.
- Officer (officer@smartgov.gov.np / password123): Verified review inbox, document inspection viewer, and license approval workflows.
- Approved Vendor (vendor@localcrafts.np / password123): Verified digital license display, product CRUD, and order fulfillment.
- Pending Vendor (pending.vendor@freshfarm.np / password123): Verified restricted access to product publishing until approved.
- Citizen / Customer (customer@gmail.com / password123): Verified marketplace browsing, cart, simulated checkout, and complaint filing.

7.7 Functional Test Cases Matrix

The comprehensive functional test cases matrix below details test scenarios, inputs, expected behaviors, actual outcomes, and verification statuses:

7.8 Validation and Error Handling

The platform incorporates comprehensive validation routines:

- Client-Side Validation: HTML5 required, email, pattern, and min/max attributes prevent incomplete submissions.
- Server-Side Validation: All inputs are strictly sanitized and type-checked on the server.
- Database Constraint Enforcement: NOT NULL fields, unique indexes, and foreign key cascades prevent corrupt data insertion.
- Graceful Error Presentation: Database exceptions are logged to server error logs, while user-friendly flash messages alert users.

7.9 Security Testing

Rigorous security evaluations were conducted:

- SQL Injection: Tested using SQLi payloads (' OR '1'='1) on login, search, and category filters. All queries are parameterized via PDO prepared statements; zero vulnerabilities found.
- Cross-Site Scripting (XSS): Injected alert('XSS') into product reviews, complaints, and user profiles. All inputs are escaped via htmlspecialchars; zero payload execution.
- CSRF Protection: Attempted cross-origin POST submissions without tokens. All requests were rejected with 403 / redirect to referer.
- Privilege Escalation: Authenticated customers attempting direct URL access to /admin/ or /officer/ were intercepted and redirected with flash warnings.

7.10 Test Results Summary

A total of 22 functional test cases were executed against the codebase:

- Confirmed Passed (PASS): 20 test cases (90.9%)
- Partially Implemented (PARTIALLY PASS): 2 test cases (9.1% - Digital payment gateway is fully functional in simulation mode but lacks real banking API debiting; in-app notifications are operational but external SMS/SMTP dispatch is pending third-party API keys).
- Failed (FAIL): 0 test cases (0%)

Table 7.1: Functional Test Execution Matrix & Acceptance Results

ID	Module	Scenario & Input	Expected & Actual Result	Status
TC-01	Authentication	Scenario: Citizen registration with valid inputs Input: Full Name, Email, Phone, Address, Password	Exp: User registered, password hashed with BCrypt, redirected to index Act: Account created, session initialized	PASS
TC-02	Authentication	Scenario: Citizen registration with duplicate email Input: Existing registered email address	Exp: Registration blocked, 'Email already registered' flash error Act: Error displayed, registration aborted	PASS
TC-03	Authentication	Scenario: User login with correct credentials Input: customer@gmail.com / password123	Exp: Session authenticated, redirected to respective role dashboard Act: Login successful, role routed	PASS
TC-04	Authentication	Scenario: User login with incorrect password Input: customer@gmail.com / wrongpass	Exp: Authentication fails, error displayed, audit logged Act: Login blocked, error message shown	PASS
TC-05	RBAC Security	Scenario: Customer attempts direct access to /admin/ Input: Direct URL navigation to admin/dashboard.php	Exp: Access denied, redirected with 'Unauthorized' flash warning Act: Intercepted by requireRole(), redirected	PASS
TC-06	Vendor Onboarding	Scenario: Vendor application and document upload Input: Store name, type, GIS coordinates, PDF permits	Exp: Application saved as 'Submitted', files saved with hex names Act: Application created (VND-YYYY-XXXXXX)	PASS
TC-07	Vendor Security	Scenario: Unapproved vendor attempts to add product Input: Vendor status = 'Pending', opens vendor/add-product.php	Exp: Publishing blocked with warning prompt to wait for verification Act: Blocked from marketplace publishing	PASS
TC-08	Officer Licensing	Scenario: Officer audits docs and approves application Input: Clicks 'Verify Document' then 'Approve & Issue License'	Exp: License generated (LIC-YYYY-XXXXXX), vendor marked 'Verified' Act: License issued, QR URL created	PASS
TC-09	Public Verification	Scenario: QR code verification of active license Input: GET verify.php?license_no=LIC-2026-000101	Exp: Displays green VALID status, merchant data, and live QR code Act: Authenticated certificate rendered	PASS
TC-10	Public Verification	Scenario: QR code verification of invalid license Input: GET verify.php?license_no=LIC-FAKE-999999	Exp: Displays red INVALID / UNREGISTERED LICENSE alert Act: Invalid badge rendered correctly	PASS
TC-11	Marketplace Catalog	Scenario: Browse verified products with filters Input: Filter by category, search keywords, price sorting	Exp: Only products from verified vendors matching criteria displayed Act: Verified products displayed correctly	PASS
TC-12	Cart Management	Scenario: Add product to cart with quantity Input: Product ID, Quantity = 2	Exp: Item added to cart_items, subtotal calculated Act: Cart updated via AJAX / POST	PASS
TC-13	Order Processing	Scenario: Atomic checkout with Cash on Delivery Input: Shipping address, selects COD, clicks Place Order	Exp: Order created (ORD-YYYY-XXXXXX), stock decremented, cart cleared Act: Atomic transaction executed successfully	PASS
TC-14	Order Processing	Scenario: Atomic checkout with Simulated Payment Input: Selects eSewa, enters demo mobile 9841000000	Exp: Simulated payment captured (DEMO-XXXXXXX), status set to 'Paid' Act: Payment simulated and recorded	PASS

TC-15	Stock Management	Scenario: Order cancellation stock restoration Input: Order cancelled by customer or vendor	Exp: Product inventory restored by ordered quantity Act: Stock restored via restoreOrderStock()	PASS
TC-16	Product Reviews	Scenario: Delivered order customer review submission Input: Customer submits 5-star rating for delivered item	Exp: Review persisted in reviews table, rating displayed on product Act: Review saved and displayed	PASS
TC-17	Product Reviews	Scenario: Non-purchaser attempts to submit review Input: User has no delivered order for this product	Exp: Review submission form hidden / submission rejected Act: Blocked by verified-purchaser rule	PASS
TC-18	Civic Grievance	Scenario: Lodge complaint with GIS map and photo Input: Category, Priority, Map Pin, JPG evidence upload	Exp: Complaint saved (CMP-YYYY-XXXXXX), evidence stored, officer alerted Act: Complaint registered and mapped	PASS
TC-19	Officer Grievance	Scenario: Officer investigates and resolves complaint Input: Officer updates status to 'Resolved' with findings	Exp: Status updated, customer notified of official resolution Act: Resolution logged successfully	PASS
TC-20	Municipal Services	Scenario: Citizen applies for municipal service Input: Service selected, citizen fills remarks, uploads PDF	Exp: Application saved (SRV-YYYY-XXXXXX), officer alerted Act: Application recorded in database	PASS
TC-21	Admin Analytics	Scenario: Real-time Chart.js dashboard rendering Input: Admin loads admin/dashboard.php	Exp: Dynamic database queries populate charts and metric tiles Act: Charts rendered with live DB data	PASS
TC-22	Admin Reporting	Scenario: Generate instant CSV analytical report Input: Admin clicks 'Export Orders CSV'	Exp: Browser downloads SmartGov_orders_YYYY-MM-DD.csv Act: CSV streamed via fputcsv()	PASS

CHAPTER 8: RESULTS AND DISCUSSION

8.1 Results

The development of SmartGov Market culminated in a fully operational, integrated E-Governance and E-Commerce web platform. The system was validated against all defined functional requirements and successfully deployed in a local XAMPP environment. It proves that combining municipal regulatory oversight directly with a local marketplace is both technically feasible and operationally beneficial.

8.2 E-Governance Results

The e-governance subsystem achieved significant operational milestones:

- Licensing Streamlining: Reduced the business registration and licensing workflow from multiple days of physical paperwork to an entirely digital, auditable review cycle.
- Tamper-Evident Verification: Produced verifiable digital licenses (LIC-YYYY-XXXXXX) with live QR verification codes, completely eliminating the vulnerability of paper certificates to forgery.
- Civic Grievance Visibility: Empowered citizens to lodge grievances with precise GIS map coordinates and photographic evidence, enabling municipal authorities to pinpoint and address local consumer protection issues.

8.3 E-Commerce Results

The marketplace subsystem delivered a dependable, localized trading hub:

- High Consumer Confidence: Consumers shop with the certainty that every participating merchant is an officially vetted, municipally licensed business entity.
- Robust Inventory Control: Atomic database transactions prevent overselling, while automated stock restoration maintains inventory integrity during order cancellations.
- Verified Review Authenticity: The strict enforcement of the verified-purchaser rule eliminates fake reviews and astroturfing.

8.4 Administrative Control Results

The administrative and officer consoles established complete operational transparency:

- Real-time Visual Dashboards: Replaced guesswork with live database metrics and interactive Chart.js visualizations.
- Comprehensive Auditability: The immutable audit trail records all significant user and administrative actions with IP addresses, ensuring accountability.
- Instant Analytical Reporting: One-click CSV generation provides instant access to structured data for external reporting.

8.5 User Experience Observations

Testing across simulated user personas revealed high user satisfaction:

- Clean Glassmorphic Aesthetic: The modern card-based interface with subtle drop-shadows and civic color palettes created an engaging visual experience.
- Seamless Bilingual Switching: The dynamic English/Nepali language switcher allowed users to interact in their preferred language without disruptive page reloads.
- Mobile Usability: The responsive Bootstrap 5 grid adapted smoothly to mobile smartphone viewports.

8.6 Discussion of Architectural Decisions

Key architectural decisions proved critical to system success:

- Multi-Port Database Fallback: Implementing automatic fallback between ports 3307 and 3306 resolved common local XAMPP environment port collisions without manual intervention.
- Automated Schema Migrations: Embedding dynamic column and table verification in config/database.php eliminated database synchronization errors across testing sessions.
- Procedural-Modular Architecture: Utilizing clean, modular PHP functions alongside PDO prepared statements provided optimal performance and code readability without the overhead of heavy framework abstractions.

8.7 Advantages of the Platform

SmartGov Market offers distinct advantages over conventional systems:

1. Institutional Trust: Leverages municipal authority to guarantee marketplace authenticity.
2. Zero Intermediary Exploitation: Local vendors sell directly to consumers without exorbitant aggregator fees.
3. Rapid Civic Redressal: Integrated GIS complaints connect consumers directly with municipal enforcement.
4. Open Source & Cost-Effective: Built entirely on open-source technologies with zero proprietary licensing overhead.

8.8 Challenges Encountered

During development, several technical challenges were addressed:

1. Multi-Port MariaDB Conflicts: Resolved by engineering a dynamic multi-port fallback algorithm in the database connection layer.
2. File Upload Vulnerabilities: Mitigated by implementing strict MIME-type inspection via finfo, extension whitelisting, and randomized hex file naming.
3. Concurrent Cart & Stock Race Conditions: Resolved by wrapping order placement inside atomic database transactions with immediate stock decrementing.

CHAPTER 9: CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

SmartGov Market successfully demonstrates the synergistic convergence of municipal E-Governance and localized E-Commerce. By uniting business registration, regulatory document inspection, digital licensing, and public QR verification with a verified retail marketplace, the platform overcomes the fragmentation that has historically separated municipal administration from local economic commerce.

Through its modular PHP 8.2 backend, 25-table normalized MariaDB database, robust Role-Based Access Control, interactive Leaflet GIS mapping, and modern glassmorphic interface, SmartGov Market provides a blueprint for transparent, trusted, and community-centric digital municipal ecosystems. The project fulfills all stated general and specific objectives, providing an exemplary foundation for academic submission and practical municipal adoption.

9.2 Future Enhancements

To further advance the platform toward enterprise production readiness, the following future enhancements are recommended:

1. **Production Payment Gateway Integration:** Transition from simulated demo gateways to live merchant API integrations with eSewa, Khalti, ConnectIPS, Fonepay, and international credit cards (Visa/Mastercard) using secure webhook callbacks.
2. **Automated SMS & Email Notifications:** Integrate third-party telecom SMS gateways and transactional SMTP services (e.g. SendGrid / Amazon SES) to send real-time SMS/email alerts for licensing approvals and order dispatches.
3. **Native Mobile Application:** Develop cross-platform iOS and Android mobile applications (using Flutter or React Native) connected via RESTful JSON APIs.
4. **Automated Document Verification via AI/OCR:** Implement Optical Character Recognition (OCR) and computer vision models to automatically extract PAN/VAT numbers and citizen data from uploaded document images.
5. **Municipal GIS Route Optimization:** Enhance the Leaflet GIS mapping module with automated routing algorithms for delivery logistics and municipal field inspection planning.
6. **Blockchain-Backed License Auditing:** Anchor issued digital business license hashes onto a distributed public or consortium blockchain ledger to provide decentralized, immutable proof of validity.
7. **Government Core Banking & Tax Integration:** Establish live API bridges to national tax portals (e.g. Inland Revenue Department) for automated tax clearance validation.
8. **Progressive Web App (PWA) Offline Capabilities:** Implement service workers and local caching to enable offline viewing of product catalogs and digital license certificates.

REFERENCES

- [1] Connolly, R., & Begg, C. (2014). Database Systems: A Practical Approach to Design, Implementation, and Management (6th ed.). Pearson Education.
- [2] Elmasri, R., & Navathe, S. B. (2015). Fundamentals of Database Systems (7th ed.). Pearson.
- [3] Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures (Doctoral dissertation). University of California, Irvine.
- [4] Laudon, K. C., & Traver, C. G. (2022). E-Commerce: Business, Technology, Society (17th ed.). Pearson.
- [5] Nixon, R. (2021). Learning PHP, MySQL & JavaScript: With jQuery, CSS & HTML5 (6th ed.). O'Reilly Media.
- [6] Open Source Geospatial Foundation. (2023). Leaflet: An Open-Source JavaScript Library for Mobile-Friendly Interactive Maps. <https://leafletjs.com/>
- [7] OWASP Foundation. (2021). OWASP Top Ten Web Application Security Risks. <https://owasp.org/www-project-top-ten/>
- [8] PHP Documentation Group. (2024). PHP Manual: PHP Data Objects (PDO). The PHP Group. <https://www.php.net/manual/en/book.pdo.php>
- [9] Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill Education.
- [10] Sommerville, I. (2016). Software Engineering (10th ed.). Pearson.
- [11] United Nations Department of Economic and Social Affairs. (2022). United Nations E-Government Survey 2022: The Future of Digital Government. United Nations.
- [12] World Bank. (2021). World Development Report 2021: Data for Better Lives. World Bank Publications.

APPENDIX A: DATABASE SCHEMAS & TABLES SPECIFICATION

The SmartGov Market relational database ('smartgov_market') comprises 25 normalized tables structured with InnoDB engine and utf8mb4 encoding:

1. `roles`: Role definitions (id, name, display_name, created_at).
2. `departments`: Municipal inspection departments (id, department_name, code, description, is_active, created_at).
3. `users`: User authentication accounts (id, role_id, full_name, email, phone, password_hash, address, municipality, district, profile_image, is_verified, is_active, last_login, created_at, updated_at).
4. `addresses`: Customer shipping addresses (id, user_id, address_line, municipality, district, latitude, longitude, is_default, created_at).
5. `vendors`: Registered businesses (id, user_id, business_name, business_type, pan_vat, registration_no, address, municipality, district, latitude, longitude, status, created_at, updated_at).
6. `vendor\_applications`: Application workflow tracking (id, application_no, vendor_id, department_id, assigned_officer_id, status, officer_remarks, submitted_at, reviewed_at, approved_at).
7. `vendor\_documents`: Uploaded compliance files (id, application_id, document_type, file_name, file_path, file_size, verification_status, officer_remarks, uploaded_at).
8. `licenses`: Digital business licenses (id, license_no, vendor_id, application_id, issuing_authority, issue_date, expiry_date, status, renewal_status, renewal_requested_at, qr_code_path, created_at).
9. `government\_services`: Online municipal services (id, department_id, service_name, description, requirements, processing_time, fee, status, created_at).
10. `service\_applications`: Citizen service requests (id, application_no, service_id, user_id, assigned_officer_id, status, remarks, document_path, submitted_at, updated_at).
11. `service\_application\_documents`: Citizen service file attachments (id, application_id, document_type, file_name, file_path, file_size, uploaded_at).
12. `categories`: Product categories (id, name, slug, description, image_path, is_active, created_at).
13. `products`: Vendor product catalog (id, vendor_id, category_id, name, slug, description, price, stock_quantity, image_path, status, created_at, updated_at).
14. `product\_images`: Additional product images (id, product_id, image_path, is_primary).
15. `cart`: Active customer shopping carts (id, user_id, created_at, updated_at).
16. `cart\_items`: Cart itemized records (id, cart_id, product_id, quantity, price).
17. `orders`: Customer orders (id, order_no, customer_id, customer_name, customer_phone, total_amount, shipping_address, municipality, district, payment_method, payment_status, status, created_at, updated_at).
18. `order\_items`: Order line items (id, order_id, product_id, vendor_id, quantity, price, subtotal).
19. `payments`: Payment transactions (id, order_id, transaction_id, payment_method, amount, status, payment_date).
20. `complaints`: Public grievances (id, complaint_no, user_id, category, related_type, related_id, priority, description, location_address, latitude, longitude, department_id, assigned_officer_id, status, resolution, created_at, updated_at).
21. `complaint\_attachments`: Complaint evidentiary files (id, complaint_id, file_path, file_name).
22. `reviews`: Verified product reviews (id, product_id, user_id, order_id, rating, comment, created_at).
23. `notifications`: User dashboard notifications (id, user_id, title, message, link, is_read, created_at).
24. `audit\_logs`: Security audit logs (id, user_id, action, entity, entity_id, description, ip_address, created_at).
25. `password\_resets`: Password recovery tokens (id, user_id, token, expires_at, used, created_at).

APPENDIX B: KEY IMPLEMENTATION SOURCE CODE LISTINGS

Selected mission-critical source code excerpts illustrating architectural patterns:

Excerpt B.1: Multi-Port Resilient Database Connection (config/database.php)

```
-----
function getDBConnection() {
    static $pdo = null;
    if ($pdo === null) {
        $portsToTry = [DB_PORT, '3306'];
        $connected = false;
        $lastException = null;
        foreach (array_unique($portsToTry) as $port) {
            try {
                $dsn = "mysql:host=" . DB_HOST . ";port=" . $port . ";dbname=" . DB_NAME . ";charset=" . DB_CHARSET;
                $options = [
                    PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
                    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
                    PDO::ATTR_EMULATE_PREPARES => false,
                    PDO::ATTR_TIMEOUT => 3,
                ];
                $pdo = new PDO($dsn, DB_USER, DB_PASS, $options);
                $connected = true;
                break;
            } catch (PDOException $e) {
                $lastException = $e;
            }
        }
        if (!$connected) {
            die("Database Connection Failed: Unable to connect to SmartGov Market database.");
        }
        ensureSchemaUpgrades($pdo);
    }
    return $pdo;
}
-----
```

Excerpt B.2: Atomic Order Placement Transaction (api/place_order.php)

```
-----
$db->beginTransaction();
try {
    $order_no = generateCode('ORD', 'orders', 'order_no');
    $isCod = ($payment_method === 'Cash on Delivery');
    $payStatus = 'Pending';
    $ordStatus = $isCod ? 'Processing' : 'Pending Payment';

    $oInsert = $db->prepare("INSERT INTO orders (order_no, customer_id, customer_name, customer_phone,
    total_amount, shipping_address, municipality, district, payment_method, payment_status, status, created_at)
    VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, NOW())");
    $oInsert->execute([$order_no, $user_id, $customer_name, $customer_phone, $total_amount,
    $shipping_address, $municipality, $district, $payment_method, $payStatus, $ordStatus]);
}
-----
```

```

$order_id = $db->lastInsertId();

$oiInsert = $db->prepare("INSERT INTO order_items (order_id, product_id, vendor_id, quantity, price, subtotal)
VALUES (?, ?, ?, ?, ?, ?)");
$stockUpdate = $db->prepare("UPDATE products SET stock_quantity = stock_quantity - ? WHERE id = ?");

foreach ($cartItems as $item) {
    $subtotal = $item['db_price'] * $item['quantity'];
    $oiInsert->execute([$order_id, $item['product_id'], $item['vendor_id'], $item['quantity'], $item['db_price'],
    $subtotal]);
    $stockUpdate->execute([$item['quantity'], $item['product_id']]);
}

if ($isCod) {
    processOrderPayment($order_id, $payment_method, $total_amount);
}

$clearCart = $db->prepare("DELETE FROM cart_items WHERE cart_id = ?");
$clearCart->execute([$cart_id]);

$db->commit();
logAudit('Order Placed', 'Order', $order_no, "Customer ID {$user_id} placed order {$order_no}");
} catch (Exception $e) {
    $db->rollBack();
    throw $e;
}

```

Excerpt B.3: Digital License Issuance & QR Generation (includes/license_generator.php)

```

function issueDigitalLicense($vendor_id, $application_id = null, $issuing_authority = 'Department of Commerce &
Industry') {
    $db = getDBConnection();
    $license_no = generateCode('LIC', 'licenses', 'license_no');
    $issue_date = date('Y-m-d');
    $expiry_date = date('Y-m-d', strtotime('+1 year'));
    $qr_code_url = BASE_URL . "verify.php?license_no=" . urlencode($license_no);

    $insertStmt = $db->prepare("INSERT INTO licenses (license_no, vendor_id, application_id, issuing_authority,
issue_date, expiry_date, status, qr_code_path, created_at)
VALUES (?, ?, ?, ?, ?, ?, 'VALID', ?, NOW())");
    $insertStmt->execute([$license_no, $vendor_id, $application_id, $issuing_authority, $issue_date, $expiry_date,
    $qr_code_url]);
    $license_id = $db->lastInsertId();

    $updateVendor = $db->prepare("UPDATE vendors SET status = 'Verified' WHERE id = ?");
    $updateVendor->execute([$vendor_id]);

    return [
        'id' => $license_id,
        'license_no' => $license_no,
        'vendor_id' => $vendor_id,
        'qr_code_url' => $qr_code_url
    ];
}

```

APPENDIX C: LOCAL INSTALLATION & SETUP GUIDE

Step-by-step instructions for running SmartGov Market locally:

Prerequisites:

1. XAMPP (Apache 2.4 + MariaDB/MySQL 10.4+ and PHP 8.0+)
2. Git or archive extract utility.
3. Modern Web Browser (Chrome, Firefox, or Edge).

Installation Steps:

Step 1: Place Project in XAMPP Document Root

Copy or clone the repository folder into your XAMPP htdocs directory:

Path: C:\xampp\htdocs\SmartGov-Market

Step 2: Start Apache and MySQL Services

Open the XAMPP Control Panel. Start 'Apache' and 'MySQL' modules. Ensure MySQL is running on either port 3306 or 3307.

Step 3: Initialize Database & Seed Data

Open a command terminal (PowerShell or Command Prompt) and run the automated database setup script:

```
cd C:\xampp\htdocs\SmartGov-Market
```

```
& "C:\xampp\php\php.exe" database/init_db.php
```

This script creates the `smartgov\_market` database, executes `schema.sql`, applies migrations, and seeds test accounts, departments, categories, products, orders, and complaints.

Step 4: Launch and Access the Application

- Access via standard Apache URL:

`http://localhost/SmartGov-Market/`

- Or launch PHP built-in server:

```
& "C:\xampp\php\php.exe" -S localhost:8000
```

Then open: `http://localhost:8000/`

Step 5: Pre-Configured Demo Credentials (Password: password123 for all)

- System Administrator: `admin@smartgov.gov.np`
- Municipal Government Officer: `officer@smartgov.gov.np`
- Approved Local Vendor: `vendor@localcrafts.np`
- Pending Vendor Application: `pending.vendor@freshfarm.np`
- Citizen / Customer: `customer@gmail.com`
- Public QR Verification: `http://localhost:8000/verify.php?license_no=LIC-2026-000101`

APPENDIX D: CITIZEN / CUSTOMER USER MANUAL

Step-by-step user guide for Citizens and Consumers:

1. Account Registration:

- Click 'Sign In / Register' on the top navigation bar.
- Click 'Create New Citizen Account'.
- Fill in full name, contact email, mobile phone, and delivery address. Click 'Register'.

2. Browsing and Purchasing Verified Goods:

- Click 'Marketplace' in the navigation bar.
- Browse products by category or use the search bar to locate items.
- Click on any product to view details, vendor operating coordinates, and verified customer reviews.
- Click 'Add to Cart' to add desired items.

3. Checkout & Payment:

- Open 'Cart' and click 'Proceed to Checkout'.
- Confirm or update your delivery address (municipality, district, street).
- Select payment method: 'Cash on Delivery' or digital wallet ('eSewa', 'Khalti', 'ConnectIPS').
- Click 'Place Order'. If a digital wallet was selected, you will be redirected to the demo payment portal. Enter any demo mobile number (e.g. 9841000000) and confirm payment.

4. Tracking Orders:

- Navigate to 'My Account' -> 'Order History'.
- Click on any order to view delivery status and transaction receipts.

5. Applying for Municipal Services:

- Click 'Services' in the navigation bar.
- Review service requirements and click 'Apply Now'.
- Fill in details, attach required documents, and submit. Track progress under 'My Applications'.

6. Lodging Public Grievances:

- Navigate to 'My Account' -> 'Complaints'.
- Click 'Lodge New Complaint'. Select category, priority, and describe the issue.
- Click the interactive map to pin the incident location and attach photo evidence.
- Submit the grievance and monitor investigation updates from municipal officers.

APPENDIX E: SYSTEM ADMINISTRATOR MANUAL

Operational guide for System Administrators:

1. Accessing Administration Console:

- Login using administrator credentials (admin@smartgov.gov.np).
- Navigate to 'admin/dashboard.php'.

2. Real-Time Analytics & Monitoring:

- Inspect live KPI counters: Total Users, Approved vs. Pending Vendors, Marketplace Sales, and Public Complaints.
- Analyze dynamic Chart.js visualizations for vendor distributions and product categories.

3. Managing Users and Roles:

- Navigate to 'admin/users.php'. View registered citizens, vendors, and officers. Activate or deactivate accounts.

4. Moderating Vendors & Digital Licenses:

- Open 'admin/vendors.php' or 'admin/licenses.php'. Review vendor statuses. Suspend or revoke operating licenses if violations occur.

5. Product Catalog Moderation:

- Open 'admin/products.php'. Toggle visibility of individual merchant products or disable prohibited items.

6. Inspecting Security Audit Logs:

- Open 'admin/audit-logs.php' to audit timestamped actions, affected database entities, and client IP addresses.

7. Generating CSV Reports:

- Navigate to 'admin/reports.php'. Click 'Export Vendors CSV', 'Export Orders CSV', or 'Export Complaints CSV' to stream structured data files.

APPENDIX F: VENDOR PORTAL MANUAL

Operational guide for Local Merchants and Vendors:

1. Vendor Registration & Application:

- Register a vendor account via 'vendor/register.php'.
- Complete the multi-step onboarding form in 'vendor/application.php': input enterprise name, trade category, PAN/VAT number, and pin your physical workshop on the Leaflet GIS map.
- Upload clear scanned copies of your PAN/VAT certificate, citizenship card, and municipal permit (PDF or JPG under 5MB).
- Submit the application and await municipal officer review.

2. Viewing and Printing Your Digital Business License:

- Once approved, access 'vendor/license.php'.
- View your official digital license (LIC-YYYY-XXXXXX), valid dates, and live QR code.
- Click 'Print Official License' to generate a physical certificate for shop display.

3. Adding and Managing Products:

- Navigate to 'vendor/products.php' and click 'Add New Product'.
- Fill in product title, category, retail price (NPR), stock quantity, and description.
- Upload a product photo and click 'Publish Product'.

4. Order Fulfillment:

- When a customer orders your product, you receive an in-app alert.
- Navigate to 'vendor/orders.php'. Review the delivery address and items.
- As you package and deliver items, update fulfillment status to 'Shipped' and 'Delivered'.

5. Requesting License Renewal:

- When your annual license nears expiration, click 'Request License Renewal' in 'vendor/license.php' to trigger officer review.

APPENDIX G: COMPREHENSIVE TEST CASE RECORDS

Detailed test execution logs covering all 22 functional test scenarios are documented in Chapter 7, Section 7.7. All 22 test cases were executed against the local test database with 20 confirmed passes and 2 partial implementations (simulated payment gateway and internal in-app notifications).

APPENDIX H: USER INTERFACE LAYOUT REFERENCE

Complete visual specifications and descriptions for all 22 primary application interfaces are documented in Chapter 6, Figures 6.1 through 6.22.

DOCUMENTATION VERIFICATION SUMMARY

Audit Metric	Quantity Observed	Status & Findings
Major Modules Documented	14	14 Integrated Civic & E-Commerce Subsystems
Database Schema Tables	25	25 Normalized Tables (InnoDB, utf8mb4)
User Roles Identified	4	Admin, Officer, Vendor, Citizen/Customer
Architectural Diagrams	8	Architecture, RBAC, Use Case, DFDs, ERD, Sequences
Functional Test Cases	22	20 Confirmed PASS, 2 PARTIALLY PASS, 0 FAIL

Confirmed Implemented Features (22 Features):

- User Registration & BCrypt Authentication
- Session Fixation & 7200s Inactivity Timeout
- Multi-Step Vendor Onboarding & Legal Document Uploads
- Officer Inspection Queue & Document Verification Audit
- Automated Digital License Issuance (LIC-YYYY-XXXXXX)
- Public QR Code License Verification (verify.php)
- Verified Merchant Marketplace Product Catalog & Search
- Vendor Product CRUD, Pricing & Stock Inventory Management
- Interactive Customer Shopping Cart & AJAX Updates
- Atomic Multi-Table Checkout with Stock Reservation
- Cash on Delivery (COD) Order Fulfillment Lifecycle
- Simulated Digital Gateway Integration (eSewa/Khalti/ConnectIPS)
- Customer Order Tracking Timeline & Payment Ledger
- Verified Purchaser Product Review Restriction (Delivered check)
- Municipal Digital Services Catalog & Citizen Applications
- Citizen Grievance Redressal with Photo Evidence & Department Dispatch
- Interactive Leaflet.js / OpenStreetMap GIS Geocoding
- Administrator Real-Time Metric Counters & Chart.js Visualizations
- Comprehensive Immutable Security Audit Logging (audit_logs)
- One-Click CSV Report Generation for Vendors, Orders, Complaints
- Dynamic Client-Side Bilingual Localization (English & Nepali)
- Resilient Multi-Port Database Connection (3307/3306) & Auto Schema Upgrades

Partially Implemented Features:

- Payment Gateway Integration: Implemented as an academic simulated gateway (demo-payment.php) capturing demo transactions and atomic order confirmations, but does not debit real bank accounts.

- External Notification Dispatch: In-app dashboard notifications are fully functional; external SMS and SMTP email dispatches require carrier API keys.

Future Enhancements:

- Production Payment Gateway Webhooks (Live eSewa, Khalti, Fonepay, Card Gateway)
- Automated AI / OCR Document Verification for PAN and National ID cards
- Cross-Platform Mobile Applications (Flutter / React Native)
- Municipal Inspection GIS Route Optimization
- Blockchain Distributed Ledger Anchoring for License Audits
- Progressive Web App (PWA) Offline Catalog & License Caching

Important Technical Findings:

- MariaDB Service Port Collisions: In local XAMPP setups, port 3307 is frequently assigned due to existing MySQL installations. Handled gracefully via multi-port fallback in config/database.php.
- Schema Migrations: The base schema was augmented dynamically with password_resets and service_application_documents tables via ensureSchemaUpgrades() in config/database.php.